Real-Time Garment Style and Colour Detection Using Confidence-Gated YOLOv8 for Smart Apparel Manufacturing

INDIVIDUAL THESIS

Jayaweera M P Y C

(IT22556370)

Individual component: Real Time Garment Style and Colour Detection with Confidence Gated Capture and Production Decision Support

Group ID: R26-IT-054

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Jayaweera M P Y C

Student ID: IT22556370



Signature: _____ Date: ____11/10/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Narmada Gamage		
Co-supervisor: Ms. Buddhima Attanayaka		

ABSTRACT

In many apparel factories, packed garments are still counted by hand, written on paper and transcribed later, so supervisors learn of delays too late to recover the lost hours. This thesis presents the author's individual research component: a real time system that recognises the style and colour of each garment at the packing table, records it only when the evidence is trustworthy, and turns the record stream into production decisions. A YOLOv8 nano detector was fine tuned on 3,033 images of four styles and runs at about 27 ms per frame on a CPU only workstation. On the held out test set it reached a precision of 0.962, a recall of 0.955 and an mAP@0.5 of 0.958. In live use, however, the detector boxed whole frames on wide views and labelled a human face as a shirt with 75.49 % confidence, failures that the benchmark could not reveal because its images share the tight, single garment framing of the training data and contain no negative examples. These failures were closed without retraining by a cascade of six interpretable gates covering region, area, the empty table, confidence, sharpness with stillness, and multiframe consensus. Colour is measured from garment pixels alone using GrabCut with a spatial prior, skin suppression in YCrCb and k means clustering in CIELAB. A FastAPI and SQLite service measures output against effective working hours net of logged downtime and reports efficiency, projected completion, overtime per day and a next day forecast, overall and per category. All 107 automated tests passed, and a bench trial confirmed end to end operation. The work shows that benchmark accuracy overstates field reliability for closed set detectors trained on cropped images, and that simple, evidence based gates can make such a detector a dependable counting instrument.

Keywords: garment recognition, YOLOv8, apparel manufacturing, colour extraction, production analytics

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Ms. Narmada Gamage, and my co supervisor, Ms. Buddhima Attanayaka, for their guidance, encouragement and patience throughout this research. Their questions pushed me to look past the headline accuracy figures and to understand what my system was really doing.

I am grateful to the staff of the apparel factory who allowed me to observe their packing area. Watching the counting process at first hand gave this project its purpose and shaped many of its design decisions.

I thank the academic and administrative staff of the Faculty of Computing at the Sri Lanka Institute of Information Technology for the knowledge and facilities provided during my degree, and the members of my research group for the discussions that helped shape this work.

Finally, I owe a deep debt to my family and friends, whose support and understanding made it possible for me to complete this work.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 1
1.3 Problem statement 3
1.4 Research gap 3
1.5 Aim and objectives 4
1.6 Research questions or hypotheses 4
1.7 Scope limitations and assumptions 5
1.8 Individual contribution 5
1.9 Thesis structure 5
2 LITERATURE REVIEW AND RELATED WORK 6
2.1 Review method and scope 6
2.2 Domain and theoretical foundations 6
2.3 Related research 8
2.4 Existing systems products and techniques 9
2.5 Gap synthesis and conceptual framework 10
3 RESEARCH METHODOLOGY 12
3.1 Research design 12
3.2 Data participants or study materials 12
3.3 Experimental or study procedure 13
3.4 Measures and evaluation metrics 15
3.5 Data analysis 16
3.6 Ethics privacy security and responsible AI 16
3.7 Reproducibility and research artefacts 17
4 PRODUCT AND SOLUTION DESIGN 18
4.1 Requirements 18
4.2 Component architecture within the complete solution 19
4.3 Design alternatives and decisions 20
4.4 Detailed design 21
4.5 Standards and professional practice 28
5 IMPLEMENTATION INTEGRATION AND TESTING 30
5.1 Development environment and technology stack 30
5.2 Component implementation 30

5.3 Integration with the other components 32
5.4 Verification and software testing 33
5.5 Deployment and operation 35
6 RESULTS AND VALIDATION 36
6.1 Research results 36
6.2 Product and technical results 40
6.3 User or stakeholder validation 41
6.4 Integration results 42
6.5 Summary of findings 42
7 DISCUSSION 44
7.1 Interpretation of findings 44
7.2 Comparison with literature baselines and existing solutions 45
7.3 Research contribution 45
7.4 Product contribution 46
7.5 Limitations and threats to validity 46
7.6 Implications 47
8 COMMERCIALISATION AND SUSTAINABILITY 48
8.1 Value proposition and target users 48
8.2 Market and competitive position 48
8.3 Business or sustainability model 48
8.4 Scalability intellectual property and compliance 49
8.5 Contribution to the Sustainable Development Goals 49
9 CONCLUSION AND FUTURE DIRECTIONS 51
9.1 Conclusion 51
9.2 Achievement of objectives 51
9.3 Individual research and product contributions 52
9.4 Future directions 52
REFERENCES 53
GLOSSARY 57
APPENDICES 58
Appendix A Complete list of automated test cases 58
Appendix B Academic integrity and AI use disclosure 59

LIST OF FIGURES

Figure 1.1: The individual component and the parts it works with 2

Figure 2.1: Conceptual framework linking garment records and production context to decision support outputs 11

Figure 3.1: A frame from a recorded packing video, selected by the earlier learned frame selector (original in colour) 13

Figure 4.1: Process architecture of the component 19

Figure 4.2: The capture gate cascade from frame to record 23

Figure 4.3: Observed confidence bands that informed the 0.85 capture threshold 24

Figure 4.4: Pipeline states reported to the dashboard 25

Figure 4.5: Two stage colour extraction design 26

Figure 4.6: Database schema of the component 27

Figure 6.1: Validation mAP@0.5 at recorded epochs during training 36

Figure 6.2: Precision, recall and mAP per class on the held out test set 37

Figure 6.3: Confusion matrix on the test set at a confidence threshold of 0.25.. 38

Figure 6.4: Pieces packed and pieces per effective hour in the worked example39

Figure 6.5: Next day forecast for the worked example 40

Figure 6.6: Time per frame on the CPU only laptop at two input sizes 40

Figure 6.7: Bench trial captures by style, confidence band and main colour 42

LIST OF TABLES

Table 2.1: Summary of related technical studies 9
Table 2.2: Comparison of existing approaches with the proposed component 9
Table 3.1: Composition of the garment detection dataset 12
Table 3.2: Detector training configuration 14
Table 4.1: Functional requirements of the component 18
Table 4.2: Nonfunctional requirements of the component 18
Table 4.3: Main design decisions and the alternatives considered 20
Table 4.4: Parameters of the capture gate cascade 23
Table 4.5: Service endpoints 27
Table 5.1: Technology stack of the component 30
Table 5.2: Structure of the automated test suite 33
Table 5.3: Representative test cases and their outcomes 33
Table 5.4: Line coverage of the tested modules 34
Table 6.1: Test set results at a confidence threshold of 0.25 36
Table 6.2: Decision support results for the worked example 39
Table 6.3: Observed weaknesses of manual counting and the corresponding features 41
Table 8.1: Benefits of the component for each user group 48
Table 8.2: Indicative cost structure for one packing station 49
Table 9.1: Achievement of the research objectives 51
Table A.1: Automated test cases and outcomes 58

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ACID	Atomicity, Consistency, Isolation, Durability
AP	Average Precision
API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
CIELAB	CIE 1976 L*a*b* colour space
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset)
CPU	Central Processing Unit
CV	Computer Vision
FN	False Negative
FP	False Positive
FPS	Frames Per Second
GPU	Graphics Processing Unit
HSV	Hue, Saturation, Value
HTTP	Hypertext Transfer Protocol
IoU	Intersection over Union
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
mAP	mean Average Precision
MJPEG	Motion JPEG
OEE	Overall Equipment Effectiveness
OT	Overtime
REST	Representational State Transfer
RFID	Radio Frequency Identification
RGB	Red, Green, Blue
ROI	Region of Interest
SDG	Sustainable Development Goal
SQL	Structured Query Language
TP	True Positive
UI	User Interface
WAL	Write Ahead Logging
WMA	Weighted Moving Average
YCrCb	Luminance, red difference and blue difference chroma colour space
YOLO	You Only Look Once

1 INTRODUCTION

1.1 Overall project context

Apparel manufacturing is one of Sri Lanka's most labour intensive industries and one of its leading export earners [1]. Each order finishes at the packing table, where completed pieces become units ready to ship. That makes it the obvious point at which to measure how much of an order is really done. In many plants, the measurement is still taken with a pen.

During a visit to the packing area of an apparel factory, the author saw operators pause roughly every fifteen minutes to count what they had packed and note the figure on a paper tally sheet. Later in the shift, someone transferred the sheets into a production dashboard. Three weaknesses were obvious. The supervisor always received the figure late. Counting and copying were separate manual steps, both prone to error, and neither left an audit trail. The tally also gave a single total for an order that in reality mixed several styles.

The group research project responds to this situation with a smart packing station in which counting, classification and reporting happen automatically and the resulting records become information a supervisor can act on during the same shift. Linking physical work to timely digital information in this way is a typical Industry 4.0 idea [2]. The work described in this thesis delivers that idea at a single packing station: a camera based recognition and capture pipeline at the folding zone, a production analytics layer, and a desktop dashboard through which the operator sets up the camera, logs stoppages and follows progress, all sharing one local data store.

1.2 Individual component context

This thesis covers the author's component: **real time garment style and colour recognition at the packing table, together with the decision support that turns recognised garments into delivery forecasts and overtime advice.** For every garment it has to answer four questions. Which style is it? Which colours are in it? Can this frame be trusted enough to count? And what does the growing record say about the order?

The component is organised in three layers. A perception service runs a YOLOv8 nano detector [3], passes its output through a series of validity gates and measures colour. A FastAPI and SQLite data service keeps the records and calculates production metrics. The third layer is a dashboard built with Electron, React and TypeScript. Figure 1.1 places the component alongside the parts it works with at the station, namely the camera, the device setup screen, the downtime log, the data service and the supervisor dashboard.

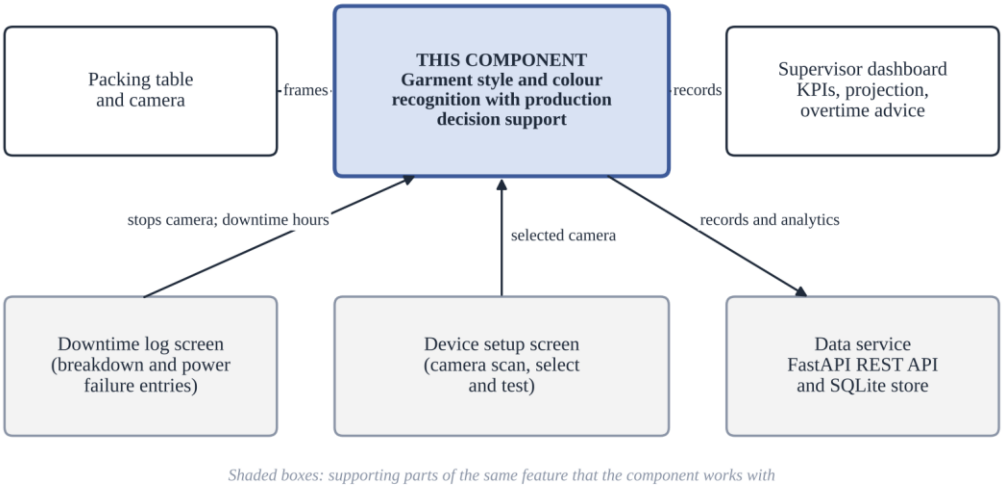


Figure 1.1: The individual component and the parts it works with

The component was built for an ordinary factory, not a laboratory. A single consumer webcam is enough, together with the type of computer that already sits beside a production line. It has to keep working while operators handle garments at their usual speed, and it has to explain its figures to supervisors with no specialist knowledge of machine learning. Nearly every decision described in the later chapters follows from these constraints.

Every figure on the dashboard depends on the garment records this component produces, so one wrong record spoils every figure that follows from it. For this reason, much of the work deals with a narrow question. When can a single recognition be trusted enough to become a permanent record?

1.3 Problem statement

Packing output is counted by hand at intervals and copied into the system later, so supervisors learn of delays too late to recover the lost hours, and they see no breakdown by style or colour. The cost of this delay is easy to underestimate. A line that falls a few percent behind its required rate each morning may look acceptable at any single moment, yet over a three week order the shortfall grows into days of delay. If the supervisor sees the gap during the shift, a small, planned amount of overtime can close it; if it only appears when the tally sheets are totalled, the remedy is usually larger, more expensive and harder on the workers.

Replacing the tally with a camera looks easy, since modern detectors score very well on benchmarks [4], [5]. The research problem appears when one asks whether that accuracy survives a working packing table. In this project the detector reached a test mAP@0.5 of 0.958, yet in live use it boxed the entire frame when shown a wide view and labelled a person's face and hair as a SHIRT with 75.49 % confidence. Neither failure is visible in the benchmark.

The problem is therefore twofold: engineering a pipeline whose counts are reliable enough to drive production decisions on a CPU only workstation, and explaining why benchmark accuracy overstates field reliability for a closed set detector trained on tightly cropped images.

1.4 Research gap

Large datasets such as DeepFashion [6] and DeepFashion2 [7], along with earlier studies of clothing parsing and style classification [8], [9], concentrate on consumer photographs and retrieval and evaluate with benchmark metrics only. In textiles, vision research has concentrated on fabric defect inspection [10], a very different task. Three gaps follow. There is little work on recognising finished garments at the packing stage, with hands, tools and table always in view. Detector studies rarely examine how a closed set model behaves on inputs absent from its test split, although work on dataset bias [11], shortcut learning [12] and benchmark generalisation [13] shows the risk is real. And no reviewed system links live recognition to interpretable, downtime aware production advice on low cost hardware.

1.5 Aim and objectives

The aim is to design, implement and evaluate a real time component that recognises garment style and colour at an apparel packing table, records each garment only when the evidence is trustworthy, and converts the records into downtime aware production metrics and delivery advice. The objectives are:

- **O1.** Train and evaluate a lightweight detector for four styles (SHIRT, T_SHIRT, TROUSER, SHORT) that runs in real time on a CPU.
- **O2.** Design a capture mechanism that turns unreliable frame level detections into a trustworthy count, with evidence for each threshold.
- **O3.** Measure garment colour from garment pixels only and name it in a perceptually meaningful way.
- **O4.** Build analytics that measure output against effective hours and give efficiency, completion, overtime and next day forecasts, overall and per category.
- **O5.** Bring the parts together in a desktop application and verify the result through automated tests and a bench trial.
- **O6.** Study how far benchmark accuracy departs from field behaviour.

1.6 Research questions or hypotheses

- **RQ1.** Can a detector at nano scale, fine tuned on a few thousand images, recognise four styles accurately enough and quickly enough on a CPU to support live counting?
- **RQ2.** Does benchmark mAP predict live reliability, and if not, which properties of the training data explain the difference?
- **RQ3.** Can simple, interpretable gates between detection and record creation close that gap without retraining?
- **RQ4.** Can colour be measured reliably from a crop that also contains table and hands?
- **RQ5.** Can rate, efficiency, completion and overtime be estimated so that downtime is separated from genuine changes in pace?

The working hypothesis was that a high benchmark score would not by itself show field reliability, and that the gap would be explained mainly by the framing and closed set nature of the training data rather than by lack of model capacity.

1.7 Scope limitations and assumptions

- Only four garment styles, seen by one fixed USB camera at one station, on a workstation with no graphics processor.
- Colour is given as up to five named colours from a palette of fifteen, and patterns are given as their component colours.
- One active order target at a time, with optional category targets and a fixed shift and break schedule.
- Evaluation includes test metrics, automated tests and a short bench trial. No long field trial against an independent manual count was carried out, so the thesis makes no claim about labour savings or counting accuracy in the field.
- The table is assumed empty when the perception service starts, because one gate takes its background snapshot from the first frame.

1.8 Individual contribution

The author prepared the dataset and trained and benchmarked the detector; designed and built the perception service with its gate cascade, capture lock and state reporting; developed the colour extraction method; built the data service, schema and analytics; developed the dashboard; wrote an automated suite of 107 tests; and documented the deployment failures that shaped the design. Earlier experiments with two learned frame selectors, later withdrawn, are reported in Section 4.3.

1.9 Thesis structure

Chapter 2 reviews the literature and sets out the conceptual framework. Chapter 3 describes the methodology, Chapter 4 the design and Chapter 5 the implementation and testing. Chapter 6 presents the results and Chapter 7 discusses them. Chapter 8 covers commercialisation and sustainability, and Chapter 9 concludes with future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review proceeded in three passes. The first covered foundational work on the techniques the component requires, namely detection, transfer learning, sharpness, segmentation, colour spaces, clustering and forecasting. The second looked at applied work near the problem, specifically garment recognition and vision in textile manufacturing. The third gathered work able to explain the behaviour observed in deployment, covering dataset bias, shortcut learning, open set recognition and calibration. Most sources were taken from IEEE Xplore, the leading computer vision conferences, the ACM Digital Library, standard textbooks and official framework documentation, and peer reviewed, widely cited work was preferred.

2.2 Domain and theoretical foundations

2.2.1 Production measurement

Apparel production is planned around a target quantity, a due date and an expected rate. Overall Equipment Effectiveness separates losses into availability, performance and quality [14]. The same principle matters here: time lost to a breakdown is an availability loss, not slow work, and mixing the two misrepresents the line. Any such measure is only as good as its input count. Barcodes and RFID can count automatically but need tags, readers and an extra scanning step, whereas a camera observes the existing work without changing it.

2.2.2 Deep learning and object detection

Modern recognition relies on convolutional networks [15] pretrained on large datasets such as ImageNet [16] and COCO [17] and then fine tuned on small task specific data. Residual networks [18] made deep backbones trainable, and MobileNetV2 [19] and MobileNetV3 [20] showed that useful accuracy is possible with little computation.

Two stage detectors such as the region proposal detector of Ren et al. [21] are accurate but slow. Single stage detectors, including SSD [22], RetinaNet [23] and the YOLO family introduced by Redmon et al. [4], predict boxes and classes in one pass

and suit real time use. YOLO has evolved through many versions [5], [24]; YOLOv8 [3] uses an anchor free head and offers a nano model of about three million parameters, small enough for CPU inference.

Transfer learning matters particularly for industrial projects, where labelled data is expensive and usually collected by the developer. A backbone pretrained on a broad dataset already responds to edges, textures and shapes, so fine tuning only needs to teach the new categories. The same mechanism carries a risk, however: a small fine tuning set can teach the model any regularity it contains, including ones that have nothing to do with the classes, such as how the images were framed.

2.2.3 Evaluation of detectors

Detectors are judged by precision, recall and average precision, where a prediction counts as correct if its Intersection over Union with a ground truth box passes a threshold. PASCAL VOC used AP at an IoU of 0.5 [25] and COCO averages over thresholds from 0.50 to 0.95 [17]; Padilla et al. [26] survey the subtleties. All these metrics describe behaviour on the evaluation set only and say nothing about inputs it does not contain.

2.2.4 Image processing foundations

A comparative study of autofocus measures [27] showed that the variance of the Laplacian is a simple, effective sharpness measure, and Pertuz et al. [28] found Laplacian based measures among the most reliable. GrabCut [29] separates foreground from background using Gaussian mixture colour models and a graph cut, and can be seeded with a mask of definite and probable labels. Chai and Ngan [30] showed that skin clusters tightly in the Cr and Cb channels of YCrCb regardless of brightness. RGB is not perceptually uniform and HSV hue is circular, so red sits at both ends; CIELAB was designed so that Euclidean distance approximates perceived difference [31], with CIEDE2000 refining it further [32]. The k means algorithm [33] summarises colours by clustering, converging only to a local optimum, which is why implementations restart it several times. Textbooks [34], [35] and OpenCV [36] provide these methods.

For colour naming, an alternative to clustering is to threshold pixels into fixed hue ranges, which is simple but brittle when a colour lies near a range boundary and awkward for red. Clustering instead lets the garment's own pixels define the groups, and only the final naming step uses a fixed palette.

2.2.5 Short horizon forecasting

Forecasting tomorrow from a few days of data is a small data problem. ARIMA [37] needs long, stable series, while simple averages and trends are competitive on short series and easy to explain [38]. A weighted moving average resists outliers but lags; a linear trend reacts quickly but overshoots, so combining them reduces error.

2.3 Related research

Garment recognition. Yamaguchi et al. [8] parsed clothing in fashion photographs and Bossard et al. [9] classified apparel by style. DeepFashion [6] offered over 800,000 annotated images for classification and retrieval, DeepFashion2 [7] added detection and segmentation, and a smaller greyscale benchmark of clothing classes also exists [39]. These works use worn or displayed garments, are evaluated by benchmarks, and target retrieval, where an occasional confident error is harmless, unlike in a production count.

Vision in textile manufacturing. Kumar's survey [10] covers fabric defect detection on moving material under controlled light, far from recognising a whole folded garment partly covered by hands. Work linking finished garment recognition to production planning is scarce.

Benchmarks versus deployment. Torralba and Efros [11] showed that classifiers transfer poorly between datasets because each carries its own biases, and Recht et al. [13] found accuracy dropping even on a carefully matched new test set. Geirhos et al. [12] described shortcut learning, where a model exploits a cue that predicts the label only within the training data. Scheirer et al. [40] showed that closed set classifiers are ill suited to environments containing unknown objects, Hendrycks and Gimpel [41] proposed confidence based detection of such inputs, and Guo et al. [42] found modern networks poorly calibrated. Sculley et al. [43] and Amodei et al. [44]

stressed that robustness and the code around the model matter as much as the model itself.

What the related work leaves open. Taken together, these studies show that garment classes can be learned, that benchmark accuracy can flatter a model, and that closed set models misbehave on unfamiliar inputs. None of them, however, examines all three issues at once in a working industrial setting, where a single confident mistake becomes a wrong production figure. Table 2.1 summarises the studies most relevant to the design and what each leaves open for the packing table problem.

Table 2.1: Summary of related technical studies

Study	Contribution	Limitation for the packing table problem
Redmon et al. [4]	Real time single stage detection	Evaluated on benchmarks only
Jocher et al. [3]	YOLOv8 with a nano variant and training toolkit	No guidance on closed set behaviour in deployment
Liu et al. [6]	Large annotated clothing dataset	Consumer imagery, not folded garments on a table
Ge et al. [7]	Detection and segmentation of clothing	GPU scale models, retrieval oriented
Rother et al. [29]	Graph cut foreground extraction	Needs good initialisation; struggles with skin and beige
Chai and Ngan [30]	Brightness independent skin map	Overlaps with some fabric colours
Pertuz et al. [28]	Comparison of focus measures	Threshold depends on scene and camera
Scheirer et al. [40]	Formal open set recognition	Requires models designed for rejection
Geirhos et al. [12]	Explanation of shortcut learning	General, not tied to an industrial task

2.4 Existing systems products and techniques

Table 2.2 compares existing approaches with the component developed here. Manual tallies cost nothing but carry delay and error. Tag based tracking is accurate but changes the work and adds recurring cost. Production planning tools interpret counts well but do not produce them. Research on clothing recognition produces strong models but targets a different setting and different hardware.

Table 2.2: Comparison of existing approaches with the proposed component

Approach	Automatic count	Style breakdown	Colour	Downtime aware rate	Extra hardware or tagging
Manual tally sheets	No	Partly	No	No	None, but labour

Approach	Automatic count	Style breakdown	Colour	Downtime aware rate	Extra hardware or tagging
					intensive
Barcode or RFID scanning	Yes	If encoded	If encoded	Not inherent	Tags, readers, scanning step
Generic production planning software	Depends on input	Depends on input	No	Sometimes	Manual or sensor input
Clothing recognition research [6], [7]	Not targeted	Yes	Attributes	No	GPU for training and inference
Fabric defect inspection [10]	No	No	No	No	Controlled light and conveyor
Proposed component (this work)	Yes, gated	Yes, four classes	Up to five named colours	Yes	One USB camera, CPU only PC

2.5 Gap synthesis and conceptual framework

The literature offers mature building blocks but no component that assembles them into a counting instrument for finished garments, that is honest about the limits of benchmark accuracy for a closed set detector, and that delivers interpretable production advice. The framework adopted here treats the system as a chain of evidence. A frame is reduced to the area where garments are expected, the detector proposes a style, and independent gates ask whether the proposal is plausible in size, distinct from the empty table, confident, sharp, still and consistent across recent frames. Only a surviving proposal becomes a record, which is then interpreted against working hours, breaks, downtime, the target and the due date (Figure 2.1). The underlying idea, drawn from work on shortcut learning and open set recognition, is that model confidence is useful but not a guarantee, so it should be combined with checks the model cannot satisfy by accident.

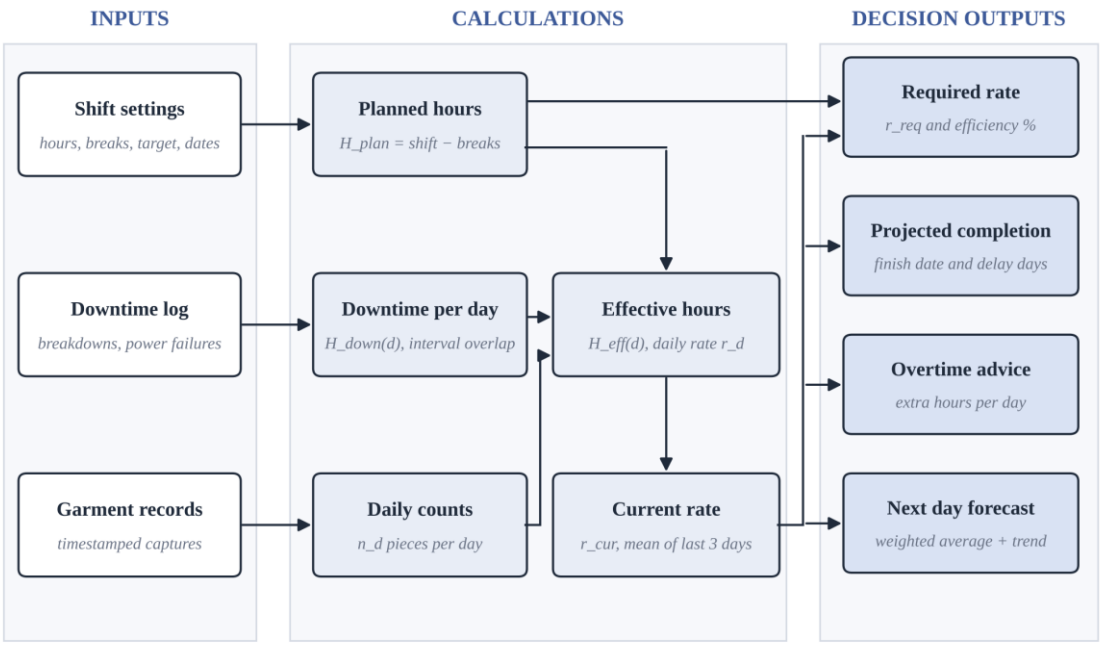


Figure 2.1: Conceptual framework linking garment records and production context to decision support outputs

3 RESEARCH METHODOLOGY

3.1 Research design

The research followed a design science approach: build an artefact for a practical problem, evaluate it, and feed the lessons back into the design. It moved through four overlapping phases: problem investigation at the packing table; dataset preparation, detector training and speed benchmarking; iterative development of the perception service, colour method, data service and dashboard; and evaluation through test set metrics, an automated test suite and a bench trial. Detector performance was measured quantitatively, deployment failures were documented as case observations tied to specific code mechanisms, and analytics were verified against hand calculations. No single form of evidence would have been enough: benchmark metrics alone hid the deployment failures, and observation alone gives no reproducible numbers.

3.2 Data participants or study materials

3.2.1 Garment detection dataset

The detector was trained on 3,033 images in four classes, annotated in YOLO format. Table 3.1 shows the training and held out test partitions.

Table 3.1: Composition of the garment detection dataset

Class	Training instances	Test instances
SHIRT	660	74
T_SHIRT	602	76
TROUSER	580	66
SHORT	628	69
Total	2,470	285 (286 images)

The classes are balanced (largest to smallest ratio 1.14), so no weighting was needed. The configuration describes a 70/15/15 split, but the folder counts imply about 81/9/9, and the thesis reports the counts found. The annotation statistics showed that almost every box is centred and covers nearly the whole image: the images are tight crops of single garments. This property turned out to be central to the findings.

3.2.2 Recorded videos, bench trial and hardware

Short videos of garments being folded at a real packing table, recorded from above with a phone, were used to develop two earlier frame selectors and the first colour method. Figure 3.1 shows a frame chosen from one of them, with hands, a tool, the table and the camera stand all in view.

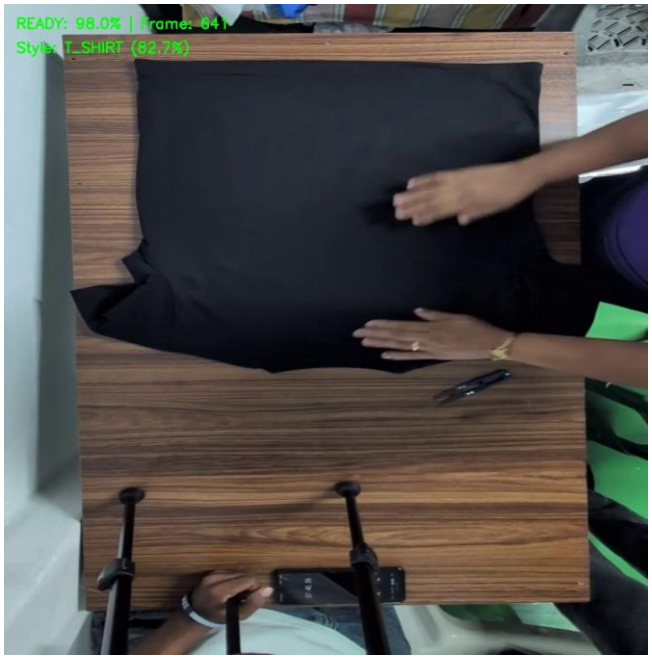


Figure 3.1: A frame from a recorded packing video, selected by the earlier learned frame selector (original in colour)

A bench trial of the complete system ran on 25 August 2026 from 11:32:42 to 11:38:12, five and a half minutes, with a target of 100 pieces, a shift from 08:00 to 18:00, a 15 minute tea break and a 45 minute lunch. Two breakdowns (0.25 h in total) and one power failure (0.03 h) were logged to exercise downtime handling. Only the author took part and no manual count was kept, so the trial demonstrates end to end operation rather than accuracy. All work ran on a laptop without a discrete graphics processor, with a standard USB webcam accessed through DirectShow.

3.3 Experimental or study procedure

Detector training. Detection rather than classification was chosen because the box is needed for the colour crop and the geometry check. YOLOv8n was fine tuned from COCO weights [17] with the Ultralytics toolkit [3] using the settings in Table 3.2. The checkpoint with the best validation mAP@0.5 was kept and evaluated once on

the test partition at a confidence of 0.25; an earlier confusion matrix at the default 0.001 had shown misleadingly low values because nearly every weak prediction was counted.

Table 3.2: Detector training configuration

Setting	Value
Architecture and initial weights	YOLOv8n, pretrained on COCO
Epochs	50
Hardware	CPU only laptop
Training time	36,214 s (about 10.06 h, 724 s per epoch)
Output checkpoint	models/best.pt (about 6.25 MB)

Speed benchmarking. Preprocessing, inference and postprocessing were timed on the laptop at input sizes of 640 and 384 pixels.

Iterative pipeline development. After each change the system was run live with garments placed, removed, moved and partly covered, and every unexpected behaviour was traced to its cause and documented beside the relevant constant. Two events were decisive. Shown an uncropped wide view, the model boxed the whole frame, which led to the region of interest and area gate; the region began with 15 % side and 20 % top margins and was later widened to a 3 % border when it cut off usable table. Later the detector labelled a person's face and hair as a SHIRT at 75.49 % and, with the threshold then at 0.75, recorded it, while every genuine capture in that session was at least 82 %; the threshold was raised to 0.85. Widening the region exposed more bare table, which led to the empty table and stillness gates.

Colour and analytics development. The first colour method assigned HSV pixels to nine fixed hue bands with a 5 % floor and a special case for red; the deployed method segments first and clusters garment pixels in CIELAB with a 1.5 % floor. The analytics were developed against a worked example: a shift from 08:00 to 17:00 with breaks of 15, 45 and 15 minutes, an order of 20,000 pieces from 24 August to 14 September, and five days of output in which the middle day lost 1.5 hours to a power failure. Every metric was calculated by hand first and the implementation tested against those values.

3.4 Measures and evaluation metrics

Precision and recall are defined from true positives, false positives and false negatives, and a prediction is a true positive when its IoU with a ground truth box exceeds the threshold.

$$P = TP / (TP + FP), \quad R = TP / (TP + FN), \quad IoU = area(B_pred \cap B_gt) / area(B_pred \cup B_gt) \qquad (3.1)$$

AP is the area under the precision and recall curve and mAP its mean over classes, reported at IoU 0.5 and averaged over 0.50 to 0.95 [17], [26]. The F1 score identifies the threshold the benchmark considers optimal. Speed is reported in milliseconds per frame. Sharpness is the variance of the Laplacian of the greyscale crop [27], and crop similarity is the mean absolute difference after resizing both crops to 64 × 64 pixels.

$$FM = (1/N)\,\Sigma\,(L_i - \bar{L})^2, \ \text{where } L = \partial^2 I/\partial x^2 + \partial^2 I/\partial y^2 \qquad (3.2)$$

The production metrics rest on effective hours: the scheduled shift less fixed breaks and less downtime overlapping the day.

$$H_eff(d) = \max(0,\ H_sched - H_break - \Sigma\ \max(0,\ \min(e_end,\ d_end) - \max(e_start,\ d_start))) \qquad (3.3)$$

The daily rate is r_d = n_d / H_eff(d), and the current rate r_cur is the mean of the last three reliable daily rates. With planned daily hours H_plan = H_sched − H_break, target T, remaining quantity R and D_left days to the due date:

$$r_req = T / (D_total \times H_plan), \ \eta = 100 \times r_cur / r_req \qquad (3.4)$$

$$D_est = \lceil R / (r_cur \times H_plan) \rceil, \quad OT = \max(0,\ R / r_cur - D_left \times H_plan) / D_left \qquad (3.5)$$

The forecast blends a weighted moving average of the last five usable rates with a least squares trend extrapolated one day [38].

$$r_pred = ½\ [\Sigma\ w_i\ r_i / \Sigma\ w_i + \max(0,\ \beta_1\ n + \beta_0)], \quad \hat{n} = \text{round}(r_pred \times H_eff(tomorrow)) \qquad (3.6)$$

"Efficiency" here means achieved pace as a share of required pace, not machine efficiency or OEE [14]. Software quality is measured by test pass rate and line coverage.

3.5 Data analysis

Detector results were analysed per class using the toolkit's validation curves, the confusion matrix at the reporting threshold and the F1 curve, with attention to background false positives and to the gap between the F1 optimum and the deployed threshold. Deployment failures were traced to properties of the data or model, and the observed confidences of genuine captures and the false positive were placed on one axis to find a separating value. Bench trial records were summarised by style, confidence band and colour. Analytics were checked deterministically against hand calculations, with edge cases such as empty history, a day lost to downtime, midnight crossing events and completed targets written as tests.

Throughout, the analysis distinguished three kinds of claim and the evidence each needs. Claims about the model, such as its accuracy, rest on the test set metrics. Claims about the software, such as the correctness of a calculation, rest on the automated tests. Claims about the deployed system, such as whether a failure was prevented, rest on live observation and are stated as qualitative. Keeping these apart prevents the strength of one kind of evidence from being borrowed for another.

3.6 Ethics privacy security and responsible AI

Participants and consent. No personal data were collected for analysis. The factory visit consisted of observing the counting process, without recording names or images of workers, and the bench trial involved only the author.

Privacy and fair measurement. Frames in the recorded videos show only the hands and arms of the person folding, were used only for development and, except for Figure 3.1, are not published. Only garment crops are stored, the live feed is not recorded, and all processing stays on the workstation. The component measures output per station and order, never per person, and excludes downtime from rates so that stoppages are not read as slow work. The face and hair false positive described in Section 3.3 also had a privacy side, since a crop of a person's head was stored as a

garment; the higher threshold and the empty table gate reduce that risk, and any such record can be deleted from the history.

Security and responsible AI. The data service listens only on the local machine, all SQL is parameterised (a test stores an injection attempt as plain data), and the desktop renderer keeps Electron's default context isolation, exposing only a small preload interface [45]. The perception service currently listens on all network interfaces, which is convenient during development but should be restricted to the local machine before deployment. The system shows its doubt as an UNCERTAIN state, displays "AWAITING DATA" rather than inventing estimates, and records known defects openly.

3.7 Reproducibility and research artefacts

The repository holds the frontend, backend, perception pipeline, training scripts, trained checkpoint and tests. Every gate threshold is a named constant with a comment explaining its origin, and the tests assert these values. Tests run against fresh temporary databases and can be repeated on any machine with the required packages, and the system starts with one command. The main limits to reproducibility are the unpublished dataset and the lack of an independent count in the bench trial.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements came from the observation of manual counting at the packing table, from what a supervisor needs to see, and from the limits of a typical factory workstation. Table 4.1 lists the functional requirements and Table 4.2 the nonfunctional ones.

Table 4.1: Functional requirements of the component

ID	Requirement
FR1	Recognise each garment in the folding zone as SHIRT, T_SHIRT, TROUSER or SHORT.
FR2	Record each garment only once, however long it stays in view.
FR3	Refuse detections that are implausible in size, look like the empty table, fall below a confidence threshold, are blurred or are still moving.
FR4	Determine the main colour and up to four secondary colours, ignoring table, hands and hair.
FR5	Store style, colours, confidence, the captured image and a server assigned timestamp.
FR6	Let the operator list, select and test the camera before monitoring starts.
FR7	Show the live feed, pipeline state, latest capture and its colours.
FR8	Let the supervisor set the order and category targets, dates, shift hours and breaks.
FR9	Let the operator log breakdowns and power failures; logging stops the camera.
FR10	Compute packed, remaining, current and required rate, efficiency, time remaining, projected completion, delay and overtime, overall and per category.
FR11	Forecast the next day's output overall and per category, stating the method used.
FR12	Present history, low confidence alerts and charts of output, style and colour.
FR13	Stop crediting garments to a target once it is reached and report when it was completed.

Table 4.2: Nonfunctional requirements of the component

ID	Requirement
NFR1	Performance: run detection at an interactive rate (about 25 frames per second or more) on a CPU only workstation.
NFR2	Reliability: a camera or perception failure must not make stored data or analytics unavailable.
NFR3	Precision over recall: when unsure, prefer missing a garment to recording a false one.
NFR4	Usability: start with one command and guide the operator through camera setup.
NFR5	Interpretability: every threshold and forecast must be explainable to a supervisor.
NFR6	Honesty: never show an estimate that the data cannot support.
NFR7	Portability: keep all data in one local file with no database server.
NFR8	Security and privacy: keep data on the workstation, parameterise SQL, store only garment crops.
NFR9	Maintainability: keep production mathematics apart from transport code and under test.

4.2 Component architecture within the complete solution

The component runs as three cooperating processes on one workstation, shown in Figure 4.1. The perception service (Flask, OpenCV and Ultralytics) owns the camera, runs detection, applies the capture gates and measures colour. The data service (FastAPI and SQLite) stores records and computes the analytics. The desktop client (Electron, React and TypeScript) presents everything to the operator and supervisor.

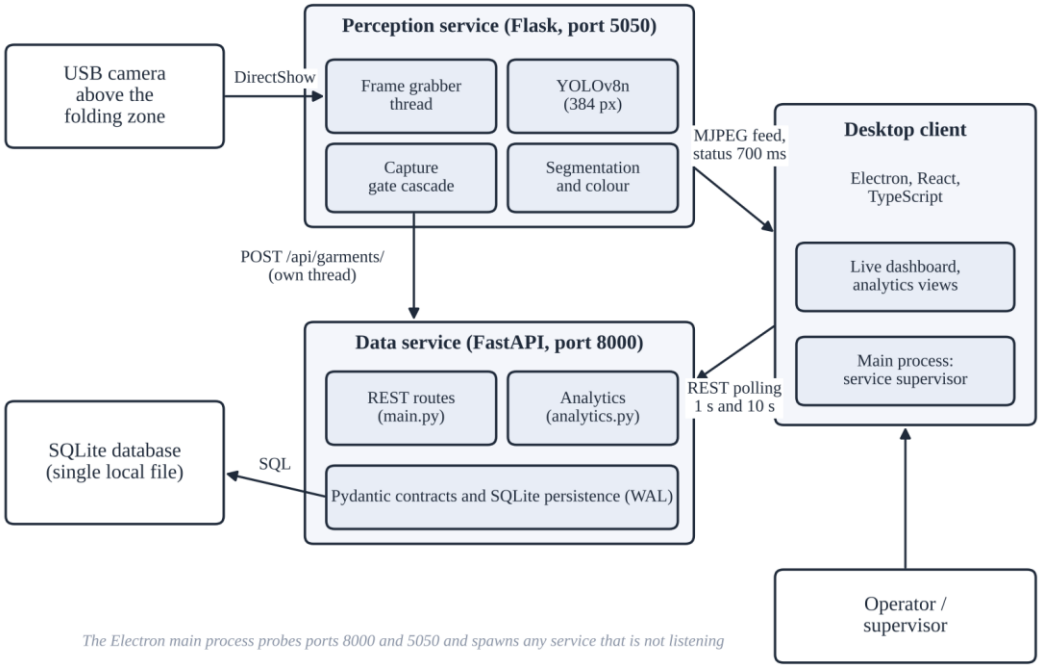


Figure 4.1: Process architecture of the component

The backend was split into two Python services for four reasons. The perception service needs PyTorch, Ultralytics and OpenCV, a heavy dependency tree, while the data service needs only FastAPI and the standard library, so each runs on its own interpreter. A camera or driver fault stops only the perception service, leaving data and analytics available (NFR2). The perception service is a long running loop that holds a device and serves an endless video stream, whereas the data service is stateless, so their concurrency models differ. Finally, either service can be restarted on its own.

The Electron main process supervises both services. It probes each service port with a 500 millisecond TCP connection and spawns the service only if nothing is listening, and it stops every child it started when the application closes. Start up is therefore idempotent and the whole system starts with one command (NFR4).

Within the station, the component supplies the garment record stream and the analytics behind the supervisor dashboard. Downtime is entered by the operator in the downtime log screen and reaches the analytics through the downtime endpoint, and the camera is chosen and tested in the device setup screen before monitoring starts. The region of interest is produced by a single function, so a different zone can be supplied later without changing the gates that follow it.

4.3 Design alternatives and decisions

Table 4.3 summarises the main design choices. The most important are explained below.

Table 4.3: Main design decisions and the alternatives considered

Concern	Alternatives	Decision	Main reason
Detector	Two stage detector, larger YOLO, classifier only	YOLOv8 nano	Real time on CPU; box needed for colour crop and geometry check
Input size	640 px, 384 px	384 px	About 27 ms per frame; more frames help the consensus gate
Best frame selection	Learned frame quality model, rule based gates	Rule based gates	Causal, cheap, explainable
Confirmation rule	Consecutive streak, rolling window	Two good of last four	One bad frame no longer erases progress
Colour space	RGB, HSV bands, CIELAB clustering	CIELAB with k means	Distance matches perception; no special case for red
Database	MongoDB, PostgreSQL, SQLite	SQLite in WAL mode	One station, no server, one portable file
Live updates	WebSockets, server sent events, polling	Polling per data type plus MJPEG	Simple and adequate for one workstation
Forecast	ARIMA, neural model, simple blend	Weighted average blended with trend	At most five data points; checkable by eye

Detector and input size. Without a graphics processor, a two stage detector built on region proposals [21] could not run in real time. YOLOv8 nano is the smallest model in its family [3], [5], and fine tuning from COCO weights [17] lets about 2,500

labelled instances teach it four new classes. Detection was preferred to classification because the box is needed for the colour crop and the area gate. At 640 pixels inference took about 52.8 ms per frame; at 384 pixels about 27 ms, roughly 37 frames per second. The smaller size was chosen because the capture rule needs two good frames within a short window, and a higher frame rate makes that more likely.

Learned frame selection versus gates. The first approach used learned models: a Keras classifier on a frozen MobileNetV2 [19] and later a MobileNetV3 Small network [20] that labelled frames as INVALID, NOT_READY or READY and kept the frame with the highest READY score above 0.74 (Figure 3.1). It was withdrawn because it was not causal: picking the best frame of a clip needs the clip to end, and a packing line never ends. It also added a second network pass on already busy hardware, to answer a question that a Laplacian filter and a counter answer cheaply.

Streak versus rolling window. Requiring several good frames in a row fails during fast handling, because one blurred or covered frame resets the count. The deployed rule keeps the last four attempts and captures when any two are good. For any per frame success probability below one, two successes out of four are more likely than two in a row, so fewer garments are missed while repeated evidence is still required.

Colour space, database and updates. The first colour method used nine fixed HSV hue bands and needed a special case for red, which sits at both ends of the hue circle. CIELAB, in which Euclidean distance approximates perceived difference [31], combined with k means [33], removes both problems. SQLite was chosen over a server database because there is one camera, one workstation and one operator; it replaced an earlier MongoDB store, and write ahead logging [46] lets the dashboard read while the pipeline writes. Polling was preferred to WebSockets: pipeline state every 700 ms, latest capture every second, summary every ten seconds, and video as one MJPEG stream.

4.4 Detailed design

4.4.1 Perception loop and frame acquisition

A background thread reads the camera continuously and overwrites one slot holding the latest frame and an increasing frame number. The detection loop reads that slot,

so it never processes a backlog of stale frames, and it skips inference when the frame number has not changed. One warm up inference on a blank image at start up moves the slow first call away from the first real garment. Four locks protect the display frame, the camera, the status record and the latest raw frame.

The loop is deliberately simple to reason about. It holds a small amount of state: the rolling attempt history, the remembered best frame, the previous crop for the stillness check, the empty table snapshot, a counter of consecutive empty frames and the capture lock. Every one of these is cleared when the camera changes, and the history, best frame and previous crop are also cleared after each capture, so evidence from one garment can never leak into the next.

4.4.2 The capture gate cascade

A single detection is never trusted. A record is created only when a detection passes every gate in Figure 4.2 and is confirmed over several frames. Table 4.4 lists the parameters.

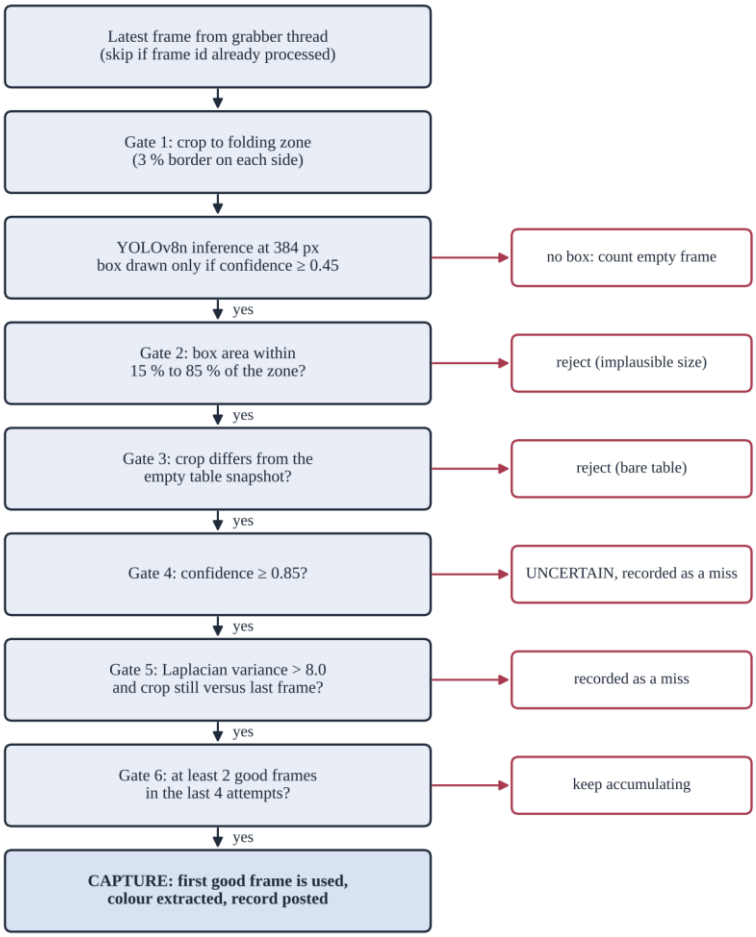


Figure 4.2: The capture gate cascade from frame to record

Table 4.4: Parameters of the capture gate cascade

Parameter	Value	Function
Region of interest	3 % border	Crop before inference, close to the training framing
INFERENCE_IMGSZ	384	Model input size in pixels
MIN_BOX_CONFIDENCE	0.45	Minimum confidence for a box to be drawn
Area gate	15 % to 85 %	Accepted box area as a share of the region
BACKGROUND_MATCH_THRESHOLD	10.0	Below this difference the crop is treated as empty table
SURE_CONFIDENCE	0.85	Minimum confidence to be eligible for capture
BLUR_THRESHOLD	8.0	Minimum Laplacian variance for a sharp crop
STILLNESS_DIFF_THRESHOLD	6.0	Below this difference from the last crop the garment is still
FAST_CAPTURE_FRAMES / CAPTURE_WINDOW	2 / 4	Good frames needed within the rolling window
CLEAR_TABLE_FRAMES /	5 / 90	Empty frames to release the lock / forced

Parameter	Value	Function
AUTO_UNLOCK_FRAMES		release

Gate 1, region of interest. Each frame is cropped to a folding zone before inference, because the model trained on tightly cropped garments boxed the whole frame when shown a wide view. The zone is drawn and labelled on the live feed, and all detection state is cleared if the camera changes.

Gate 2, area. The highest ranked box must cover 15 % to 85 % of the zone. Larger boxes are the full frame failure and are rejected whatever their confidence; smaller ones cannot be a folded garment.

Gate 3, empty table. Widening the zone exposed more bare table to a detector with no "not a garment" class. The service keeps a snapshot of the empty zone, taken at start up and refreshed whenever the zone is confirmed empty, and rejects any crop whose mean colour difference from the snapshot is below 10.0.

Gate 4, confidence. A detection below 0.85 becomes UNCERTAIN, is shown in orange, is reported to the dashboard and counts as a miss. As Figure 4.3 shows, genuine captures in the relevant session were all at or above 0.82, while the face and hair false positive was 0.7549. The threshold reflects NFR3: a missed garment can be shown again, but a false record corrupts every figure built on it.

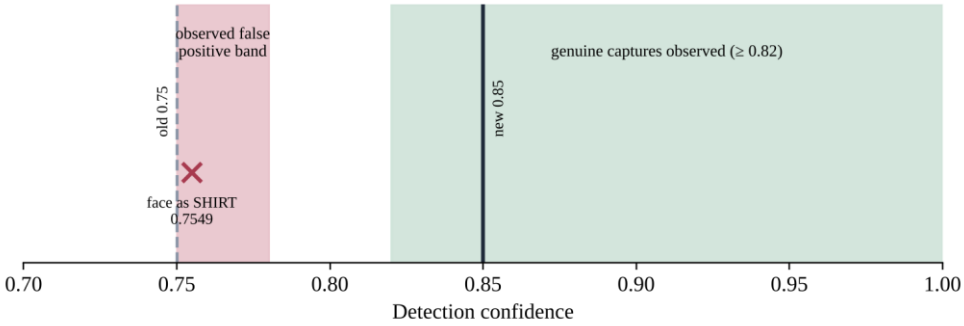


Figure 4.3: Observed confidence bands that informed the 0.85 capture threshold

Gate 5, sharpness and stillness. The variance of the Laplacian of the crop must exceed 8.0, computed in 64 bit floating point so negative responses are kept. The threshold was lowered from 15.0 because garments still settling in the operator's hands were rejected too long. Because a fast shutter can make a sliding garment look sharp, the crop must also differ from the previous frame's crop by less than 6.0.

Gate 6, consensus. Each attempt adds true or false to a window of four, and a capture fires when two are true. The first good frame is remembered at the moment it occurs and is the one stored, so the image, style and confidence always come from a frame that actually passed the checks; earlier, a capture could be triggered on a later blurred frame.

4.4.3 Capture lock and pipeline states

After a capture a lock prevents double counting and the screen asks the operator to remove the garment. The lock is released after more than five consecutive empty frames, or forcibly after 90 frames so the line cannot stall. A hand briefly passing over the garment does not release it. Figure 4.4 shows the states reported to the dashboard.

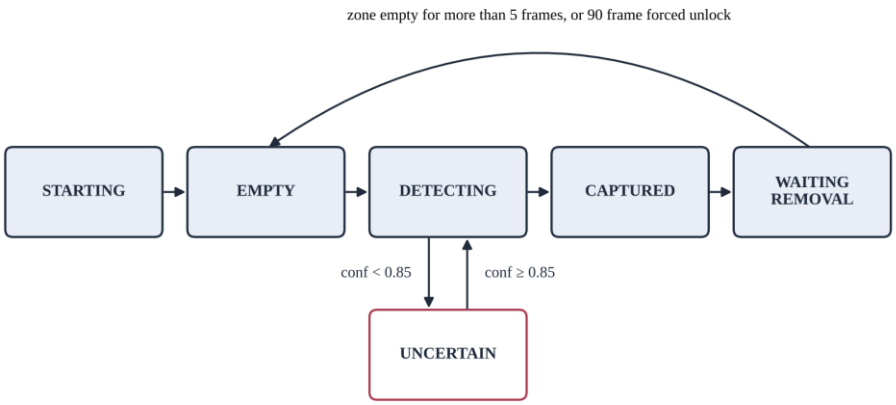


Figure 4.4: Pipeline states reported to the dashboard

4.4.4 Colour extraction design

Colour is measured from garment pixels only, in the two stages of Figure 4.5.

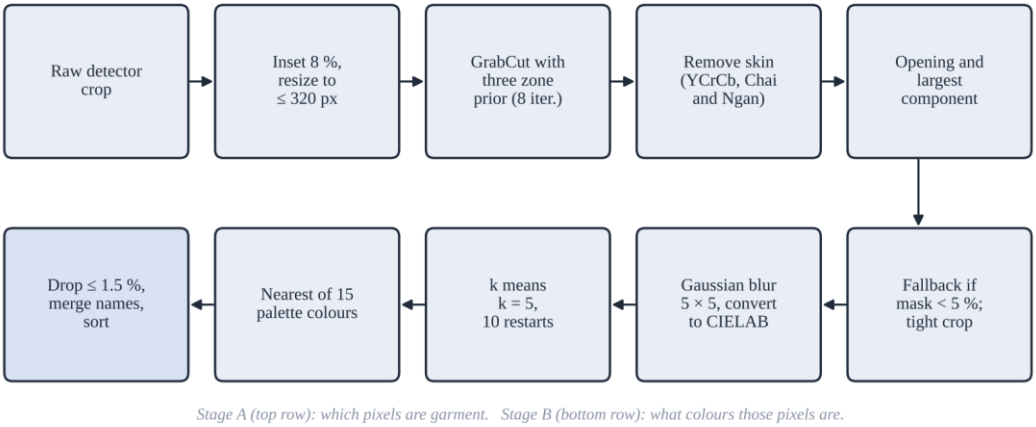


Figure 4.5: Two stage colour extraction design

In Stage A the crop is inset by 8 % on each side, where box edges are least precise, and reduced to at most 320 pixels on its longer side. GrabCut [29] is seeded with a three zone mask: pixels outside a 10 % border are probable background, pixels inside an 18 % inset are definite foreground, and the band between is left to the algorithm, which runs eight iterations. A garment that has passed the area gate fills most of its crop, so this strong seed gives a better colour sample than a plain rectangle. Skin pixels, with Cr between 133 and 173 and Cb between 77 and 127 [30], are then removed, since a resting hand and beige fabric can share a mixture component. An elliptical opening sized at 3.5 % of the shorter side removes fingers and hair, and only the largest connected component is kept. If less than 5 % of the pixels survive, the whole crop is used instead.

In Stage B the image is smoothed with a 5 × 5 Gaussian filter, converted to CIELAB, and only foreground pixels are clustered with k means (k = 5, ten restarts). Each centre is named as the nearest of fifteen palette colours by squared Euclidean distance; clusters of 1.5 % or less are dropped, clusters with the same name are merged, and the largest becomes the main colour. Failures return UNKNOWN. The mask is never displayed; the stored and shown image is always the untouched detector crop.

4.4.5 Data model and interfaces

The data model has four tables, shown in Figure 4.6. The settings and device tables are singletons enforced by CHECK (id = 1), which also allows an atomic insert or

update. Only the garment timestamp and the downtime interval bounds are indexed, because every analytical query filters on them. Breaks and category targets are stored as JSON text because they are always read with the rest of the settings. Images are stored inline so that each capture is atomic, and a count_since column marks the start of the current counting cycle separately from the order dates.

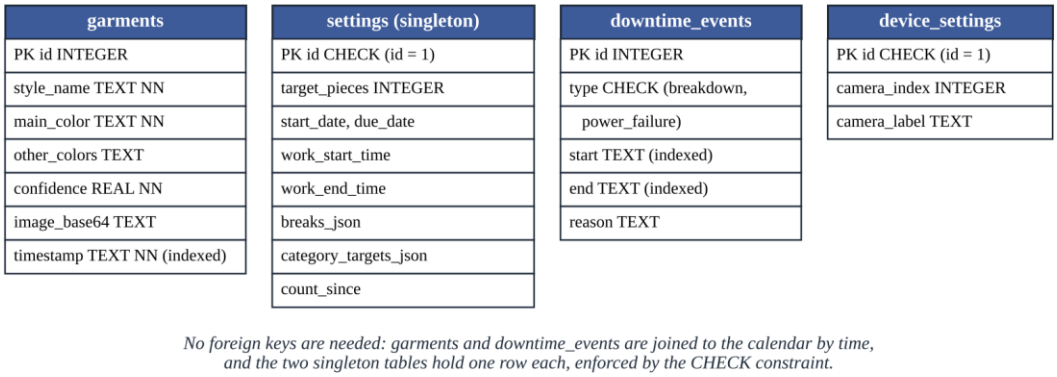


Figure 4.6: Database schema of the component

Table 4.5 lists the endpoints. Request bodies are validated by Pydantic models before any handler runs, and enumerated values such as the downtime type are constrained in both the model and the database.

Table 4.5: Service endpoints

Method	Path	Purpose
GET, POST, DELETE	/api/garments/	History; store a capture; clear history
GET	/api/garments/latest	Most recent capture, or null
GET, PUT	/api/settings	Target and shift configuration
POST	/api/settings/reset-progress	Start a new counting cycle
GET, PUT	/api/device/camera	Selected camera
POST, GET	/api/downtime	Log or list interruptions
GET	/api/stats/daily, /api/stats/summary	Daily counts; decision summary
GET	/api/predict/next-day	Next day forecast
GET	:5050/video_feed, :5050/api/pipeline/status	Live stream; pipeline state
GET, POST	:5050/api/device/cameras, test-camera, stop-camera	Enumerate, switch and release the camera

4.4.6 Analytics and interface design

All metrics come from two pure functions, one for the decision summary and one for the forecast. Each receives a target, a packed count and a rate history, so the same mathematics serves the whole order and each of the four categories. Because the class names are identical in the model, the database, the request models and the interface, a category filter is a simple equality test.

Several refinements came from real use. Today's effective hours are measured against time elapsed since the shift started, which stopped efficiency from appearing to drift upward through the day. Days with under five minutes of effective time are ignored in averages. When no reliable daily rate exists, a live pace is taken from the last five record timestamps. When a target is reached, the count is frozen, the completion time is found from the record that reached it, and extra garments are reported as overrun.

The dashboard groups its views as Setup, Monitor and Admin. It opens on device setup, whose Continue button stays disabled until the camera test passes. The live dashboard shows packed and remaining pieces, estimated time remaining, projected delivery, suggested overtime, average cycle time and order progress, with a breakdown for each garment category. Colours carry consistent meanings (green on track, amber attention, red delay); suggested overtime turns red above three hours per day. When an order is completed, an order summary reports the days and hours taken, whether the order finished ahead of or behind schedule, and the achieved efficiency. Numbers use a monospaced typeface, and the video panel sizes itself from the real frame dimensions so the folding zone is never cropped.

4.5 Standards and professional practice

The backend keeps request schemas, persistence, domain logic and transport in separate modules, and translates domain errors into conventional HTTP codes: 400 for an invalid downtime interval, 404 for missing configuration and 422 for invalid input. FastAPI generates OpenAPI documentation automatically [47]. Database access follows SQLite guidance: write ahead logging, one connection per request, commit only on success, parameter binding throughout [46], and a forward only migration that is safe to repeat. The desktop client keeps Electron's context isolation

and exposes only a vetted preload interface [45], and timestamps use ISO 8601. The work was tracked in Microsoft Teams and Planner with a weekly log book, thresholds were documented beside the constants they control, and known defects were recorded in the test suite.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component was developed on a Windows laptop without a discrete graphics processor, the same class of machine it targets. Table 5.1 lists the technologies used.

Table 5.1: Technology stack of the component

Layer	Technology	Reason for selection
Desktop shell	Electron 33	Native window and process supervision
User interface	React 18, TypeScript 5.7, Vite 5	Component model, typed API payloads, fast rebuilds
Styling and charts	Tailwind CSS 4, Chart.js 4	Token based design; line, bar and doughnut charts
Data service	FastAPI, Uvicorn, Pydantic	Automatic validation and API documentation
Persistence	SQLite in WAL mode	Zero configuration, transactional, one file
Perception service	Flask, OpenCV, NumPy	Streaming, image processing, clustering
Detection	Ultralytics YOLOv8n on PyTorch	Small, fast detector with fine tuning support
Camera enumeration	DirectShow (pygrabber)	Same subsystem that opens the cameras
Testing	pytest, FastAPI TestClient, coverage plugin	Unit and integration tests with coverage
Project management	Microsoft Teams and Planner	Task tracking and weekly log book

The repository has a frontend folder (the Electron application and its service supervisor), a backend folder (models, database, analytics and main modules), a resources folder (the perception service, training scripts, the trained checkpoint and a separate virtual environment) and a tests folder.

5.2 Component implementation

Detector. The model was trained with the Ultralytics interface and the best checkpoint saved as models/best.pt (about 6.25 MB). The perception service loads it once at start up and exits with a clear message if it cannot.

Perception loop. The loop runs in a background thread while the Flask server, started with threading enabled, serves the endless video stream alongside the other routes. Each cycle checks for an active camera (sleeping 200 ms if there is none, for example after a breakdown has been logged), recomputes the zone if the camera

changed, and skips the cycle if no new frame has arrived. The core of the gate logic is shown below in condensed form.

```
if MIN_AREA <= box_area <= MAX_AREA:
    crop = frame[y1:y2, x1:x2]
    if mean_abs_diff(crop, background[y1:y2, x1:x2]) >= 10.0 and not
locked:
        if conf < 0.85:
            history.append(False)              # UNCERTAIN
        else:
            good = laplacian_var(crop) > 8.0 and mean_abs_diff(crop,
prev) < 6.0
            history.append(good)               # deque(maxlen=4)
            if good and best is None:
                best = (crop, style, conf)
            if sum(history) >= 2:
                capture(best)
```

On capture, the stored crop is segmented, its colours extracted, the raw crop encoded as base64 JPEG, and the record posted to the data service on its own thread with a five second timeout, so a slow backend never delays frame processing. The streaming generator sleeps 50 ms when no frame is ready; without this pause an idle viewer held a request thread at full CPU load.

Camera management. The service owns exactly one camera behind a lock. Previously the setup test and the detection loop opened separate captures, so testing a camera did not change what the dashboard showed. Now testing a camera switches the live one, and the camera activity flag is computed fresh on every status request.

Colour functions. Inset, skin mask, mask cleaning, GrabCut foreground, segmentation and clustering are separate small functions, which made each testable with synthetic images. None of them raises on bad input; they return UNKNOWN.

Data service. All database access goes through the context manager below.

```
@contextmanager
def get_connection():
    conn = sqlite3.connect(DB_PATH, check_same_thread=False)
    conn.execute("PRAGMA journal_mode=WAL")
    conn.execute("PRAGMA foreign_keys=ON")
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()
```

It opens a connection for each request with WAL enabled, commits only if the enclosed block succeeds, and always closes the connection. The database path is resolved relative to the source file, because Electron launches the service from another directory and a relative path would silently create an empty database. Settings are saved with one INSERT ... ON CONFLICT DO UPDATE statement, and saving moves count_since to the current moment so that a new target starts from zero, while the all time history chart stays unscoped. Downtime overlap is computed per day from events selected by a half open interval test, which handles events that cross midnight.

Desktop client. The Electron main process searches for a Python interpreter that can actually import FastAPI and Uvicorn, because a bare command name did not always resolve when Electron spawned it, and starts the perception service with its own virtual environment. In the React renderer, all network access sits in one typed module; read functions return empty values on error so a backend hiccup never crashes a view, while write functions report failures to the operator. The one second poll compares the latest record identifier with a stored reference, so the view updates only on a genuinely new capture. Deleting history needs a two step confirmation, logging downtime stops the camera, cards show "AWAITING DATA" when no rate can be computed, and the forecast is drawn as a separate dashed series labelled with its method.

5.3 Integration with the other components

The perception and data services meet only at the JSON contract of the garment endpoint, so the perception code contains no SQL. The desktop client talks to both services over local HTTP exactly as a browser would, and the Electron supervisor brings them together so the whole system starts with npm start. Downtime entered in the downtime log, camera choices made in device setup and garment records posted by the perception service all pass through validated endpoints of the same data service, and the four class names are a shared vocabulary across model, database and interface. Integration was verified by the integration test suite, which exercises the data service end to end over HTTP, and by the bench trial in Section 6.4, which ran camera, perception, data service and dashboard together.

5.4 Verification and software testing

5.4.1 Test strategy and suite

Testing focused on parts whose correctness can be decided exactly: the analytics, the persistence layer, the HTTP interface and the image processing functions. Each test uses a fresh temporary SQLite file built by the application's own initialisation code, and the vision tests import the real perception module with only hardware and network libraries stubbed. Table 5.2 summarises the suite.

Table 5.2: Structure of the automated test suite

Suite	Tests	Focus
Analytics unit tests (UT_A)	31	Hours, downtime overlap, effective hours, rates, rounding, overtime, completion, forecast
Database unit tests (UT_D)	14	Tables, indexes, constraints, singletons, WAL, rollback, idempotent migration
Vision unit tests (UT_V)	36	Zone and area gate, sharpness, similarity, skin mask, segmentation, colour naming, consensus
Integration tests (IT)	26	Validation, persistence, timestamps, injection, settings, downtime, summary, forecast, completion
Total	107	

5.4.2 Representative test cases

Table 5.3 lists selected cases; the complete list is in Appendix A.

Table 5.3: Representative test cases and their outcomes

ID	Scenario	Expected result	Outcome
UT_A07	Power failure from 22:00 to 02:00	2 h on each day	Pass
UT_A13	Today's hours early in the shift	Capped to elapsed time	Pass
UT_A21	25.87 days of work remaining	Rounded up to 26	Pass
UT_A22	No rate history	Estimates null, not guessed	Pass
UT_A25	Packed count exceeds target	Frozen at target; overrun reported	Pass
UT_D04	Second settings row	Rejected by CHECK	Pass
UT_D13	Older database without count_since	Column added by migration	Pass
UT_V04	Box covering the whole zone	Rejected by area gate	Pass
UT_V07	Blurred image	Fails sharpness gate	Pass
UT_V25	Navy garment with white panel	Main NAVY, WHITE secondary	Pass
UT_V30	Attempts bad, good, bad, good	Capture on fourth	Pass

ID	Scenario	Expected result	Outcome
		attempt	
IT_12	SQL injection string as style	Stored as plain data	Pass
IT_22	Middle day with 1.5 h power failure	Its rate matches other days	Pass

The tests were written around the behaviours a supervisor relies on rather than around individual functions. For example, the integration tests seed five days of production with a power failure on the middle day and then check, through the same HTTP calls the dashboard makes, that the rate, efficiency, estimated days and overtime match the worked example. The vision tests use synthetic images with known properties, such as a sharp checkerboard, a blurred copy of it, a flat grey patch and solid colour panels, so that each expected outcome is unambiguous. Several tests also guard documented values: if a threshold such as SURE_CONFIDENCE or BLUR_THRESHOLD is changed in the code without the documentation being updated, a test fails.

5.4.3 Execution, coverage and defects

All 107 tests passed in about three seconds. Line coverage, in Table 5.4, is 96 % to 100 % for the data service. The perception module reaches 38 % because much of it is the live camera loop, enumeration and streaming server, which need hardware and were checked in the bench trial; its image processing functions are all exercised.

Table 5.4: Line coverage of the tested modules

Module	Statements	Missed	Coverage
backend/analytics.py	220	8	96 %
backend/database.py	27	0	100 %
backend/main.py	123	4	97 %
backend/models.py	34	0	100 %
resources/live_webcam_pipeline.py	487	303	38 %
Total	891	315	65 %

Several defects were found and fixed: efficiency drift for the current day (guarded by UT_A13), a migration that could never add a new column to an existing table (UT_D13), and blurred stored images caused by capturing on the wrong frame. One defect was recorded instead of silently fixed. The forecast pairs the weights 5, 4, 3, 2, 1 with rates stored oldest first, so the oldest day gets the largest weight. Test

UT_A31 documents this behaviour and will fail when the one line fix is applied; its effect is quantified in Section 6.1.4.

5.5 Deployment and operation

Deployment needs the frontend packages, a Python interpreter with FastAPI and Uvicorn, and the perception virtual environment with PyTorch, Ultralytics and OpenCV. The operator then starts the system with one command, tests the camera in device setup, and continues to the live dashboard. Each garment is placed in the folding zone and removed once the status shows it was captured; breakdowns and power failures are logged with start and end times. The supervisor sets targets, dates, shift hours and breaks, and reads progress, projections, overtime advice and forecasts from the dashboard. The database was about 35 MB during development because images are stored inline; for long term use, storing images as files would keep queries fast, and the development servers without authentication would need replacing for a networked deployment (Section 7.5).

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Training and held out test results

Validation mAP@0.5 rose from 0.720 after the first epoch to 0.954 after ten and then levelled off near 0.985 (Figure 6.1). The best checkpoint, at epoch 41, reached precision 0.9484, recall 0.9676, mAP@0.5 0.9872 and mAP@0.5:0.95 0.9073; the final epoch gave precision 0.9574 and recall 0.9676. The gap of about 0.08 between the two mAP figures shows that the remaining error lies mostly in box tightness rather than class choice.

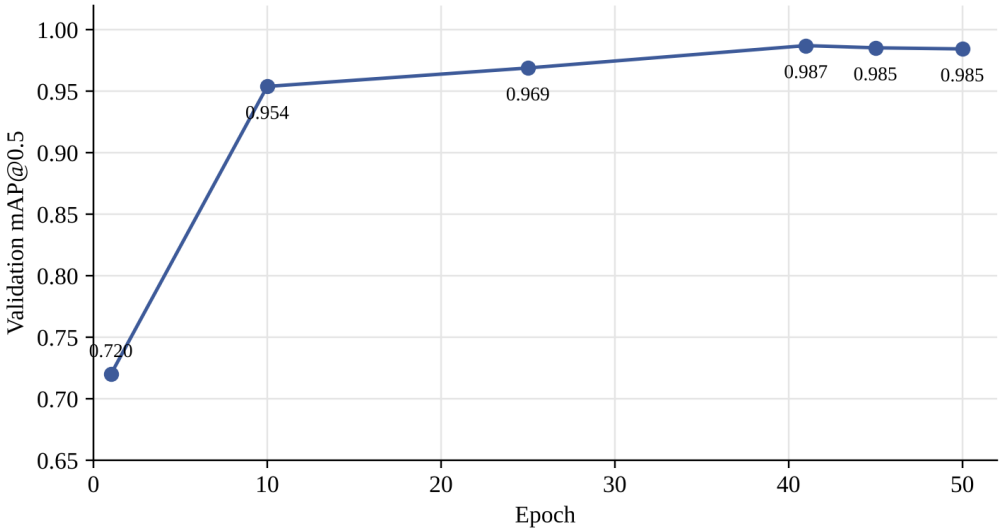


Figure 6.1: Validation mAP@0.5 at recorded epochs during training

On the 286 image test partition at a confidence of 0.25, the model achieved the results in Table 6.1 and Figure 6.2: precision 0.962, recall 0.955 and mAP@0.5 0.958 overall. In benchmark terms, RQ1 is answered positively.

Table 6.1: Test set results at a confidence threshold of 0.25

Class	Instances	Precision	Recall	mAP@0.5	mAP@0.5:0.95
SHIRT	74	0.975	0.919	0.938	0.874
SHORT	69	1.000	0.925	0.945	0.866
TROUSER	66	0.942	0.976	0.991	0.875
T_SHIRT	76	0.932	1.000	0.958	0.910
All	285	0.962	0.955	0.958	0.881

Differences between the classes are small but informative. TROUSER had the highest mAP@0.5 (0.991), which fits its long, distinctive outline even when folded. SHIRT had the lowest recall (0.919), while T_SHIRT reached a recall of 1.000 with the lowest precision (0.932). Read together, these figures suggest that the model leans towards T_SHIRT when a shirt's distinguishing details are hidden, which the confusion matrix below confirms. SHORT reached the highest precision (1.000 at the reported operating point), although some shorts were missed or taken for trousers.

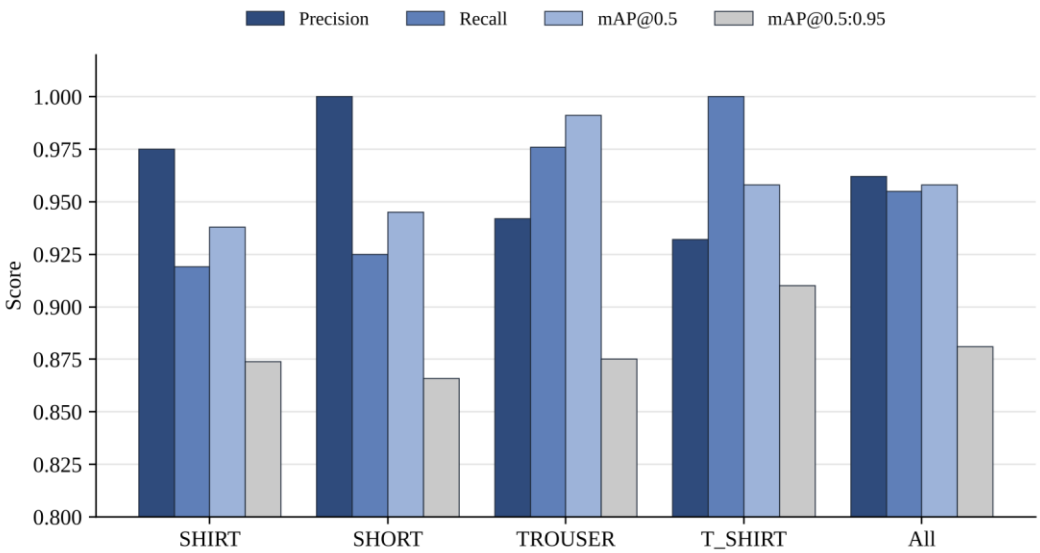


Figure 6.2: Precision, recall and mAP per class on the held out test set

6.1.2 Error structure

The confusion matrix in Figure 6.3 shows 67 of 74 shirts, 65 of 69 shorts, 65 of 66 trousers and all 76 T shirts classified correctly. The largest error is asymmetric: seven shirts were predicted as T shirts and none the other way, which is plausible because a folded shirt seen from above can hide its collar and placket. Shorts and trousers were confused four times, and one short was predicted as a T shirt. Most important for this thesis are fifteen background false positives, about 5 % of predictions, where a garment was predicted with no garment present; no ground truth garment was missed. These are an early, mild sign of the closed set behaviour seen later in live use.

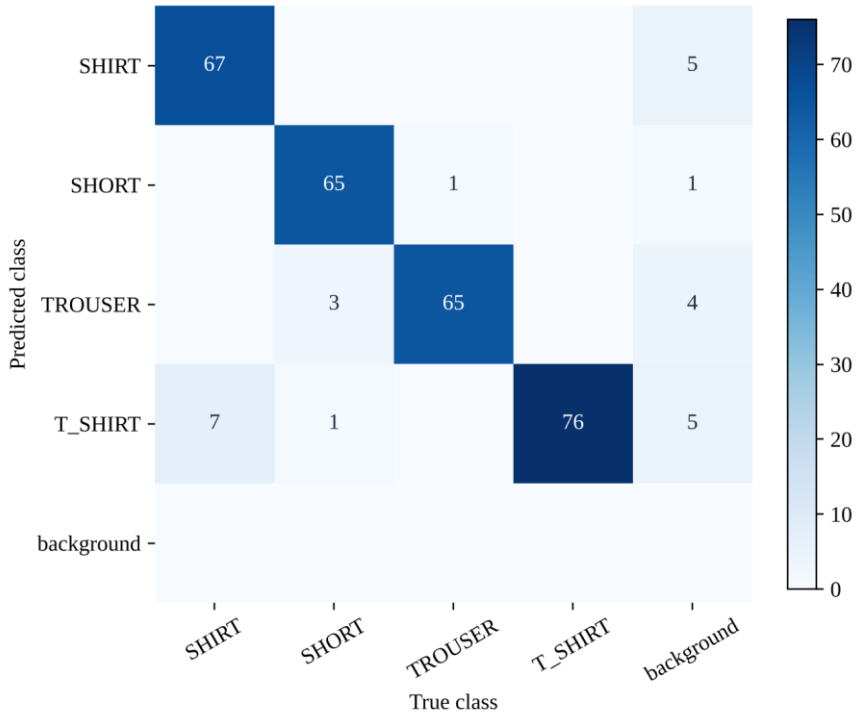


Figure 6.3: Confusion matrix on the test set at a confidence threshold of 0.25

The F1 score peaked at 0.96 at a confidence of 0.665, so the benchmark alone would recommend a threshold near 0.67. The deployed threshold is 0.85, where recall read from the plotted curve is roughly 0.90 against about 0.97 at very low thresholds; this value is approximate.

6.1.3 Analytics on the worked example

In the worked example (Figure 6.4) the middle day packed only 500 pieces but lost 1.5 hours to a power failure, so its 6.25 effective hours give a rate of 80.00 pieces per hour, exactly the same as the first day. The effective hours model attributes the low count to lost time, not slower work.

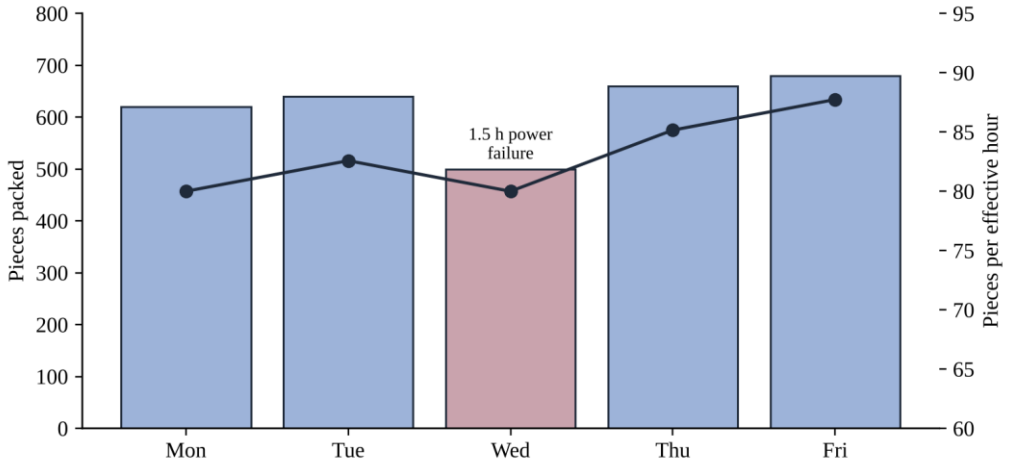


Figure 6.4: Pieces packed and pieces per effective hour in the worked example

Table 6.2 lists the decision figures. Each was calculated by hand and returned identically by the implementation, and integration test IT_21 reproduces the scenario through the HTTP interface.

Table 6.2: Decision support results for the worked example

Quantity	Value	Basis
Planned hours per day	7.75 h	9.00 h shift less 1.25 h breaks
Total packed / remaining	3,100 / 16,900	Target 20,000
Current rate	84.30 per hour	Mean of 80.00, 85.16, 87.74
Required rate	122.89 per hour	20,000 over 21 days of 7.75 h
Efficiency	68.6 %	84.30 over 122.89
Estimated days to target	26 days	25.87 rounded up
Projected completion / delay	23 September / 9 days	Due 14 September
Suggested overtime	4.04 h per day	68.72 h shortfall over 17 days
Cycle time / progress	42.7 s per piece / 15.5 %	3,600 over 84.30; 3,100 over 20,000

Rather than only reporting a nine day delay, the system advises about four extra hours per day from today. For the forecast (Figure 6.5), the trend through the five rates has slope 1.806 and intercept 79.484, giving 88.52 for the next day. With weights applied newest heaviest as intended, the weighted average is 84.30, the blend 86.41 and the forecast 670 pieces. With the weights in the order currently shipped, the forecast is 660 pieces, a difference of about 1.5 % that changes no conclusion but is a genuine defect (Section 5.4.3).

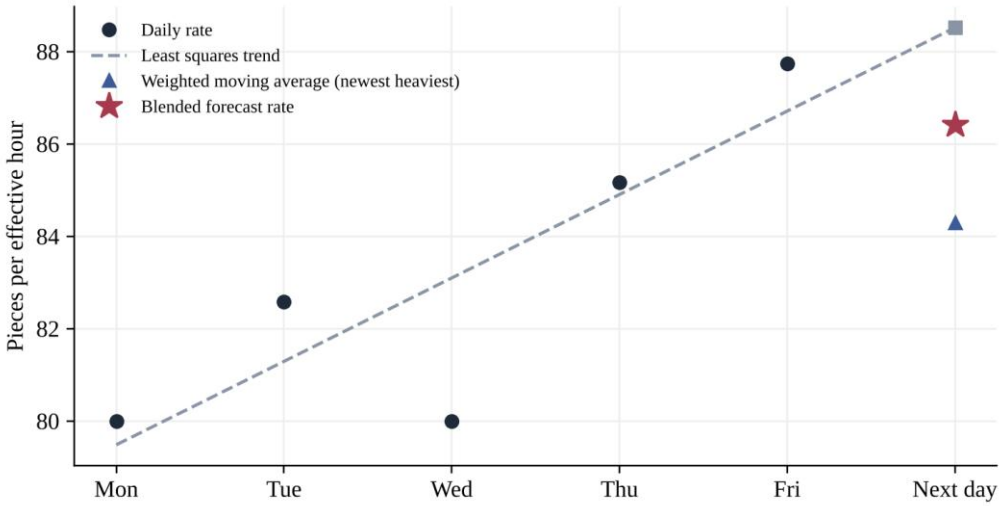


Figure 6.5: Next day forecast for the worked example

6.2 Product and technical results

Speed. At 640 pixels a frame took about 1.1 ms of preprocessing, 52.8 ms of inference and 0.5 ms of postprocessing, near 19 frames per second; at the deployed 384 pixels, about 27 ms or 37 frames per second (Figure 6.6). NFR1 is met with margin.

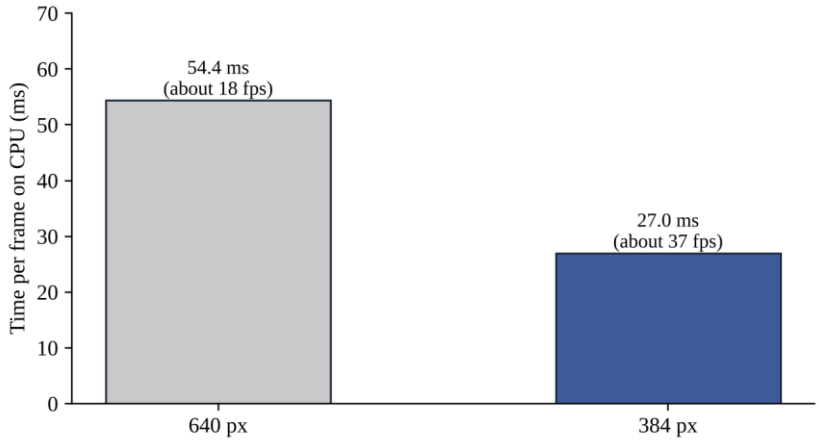


Figure 6.6: Time per frame on the CPU only laptop at two input sizes

Deployment failures. Two failures in live use could not be predicted from the benchmark. Shown an uncropped table, the detector returned one confident box covering the whole frame. Because nearly every training box covers the whole image, the model learned that the right box is almost always the full frame, and since the test images share that framing, a full frame prediction scores an IoU close to one; the benchmark measures classification far more than localisation. Separately, the

model labelled a face and hair as a SHIRT at 75.49 %, because a four class model with no "not a garment" class must assign every input to one of its classes.

Effect of the gates. Each gate targets one failure: the region of interest supplies familiar framing, the area gate rejects any box above 85 % of the zone regardless of confidence, the empty table gate rejects bare table, the confidence gate rejects the face detection at 0.7549 and shows it as UNCERTAIN, and the sharpness, stillness and consensus checks require a settled, repeated view. After the gates were introduced, neither failure produced a record in live testing. This evidence is qualitative, since rejected detections were not formally counted. The 0.85 threshold sits well above the F1 optimum of 0.665, deliberately trading recall for fewer false records (NFR3).

Colour and software quality. In the vision tests, solid patches were named NAVY, RED, WHITE and BLACK correctly, a navy garment with a white panel returned NAVY with WHITE as secondary, stray pixels under the 1.5 % floor were ignored and empty input returned UNKNOWN. All 107 automated tests passed, with 96 % to 100 % coverage of the data service modules.

6.3 User or stakeholder validation

Table 6.3 maps the weaknesses observed at the factory packing table to features of the component. No structured usability study with factory supervisors and operators was carried out within the project period; this is future work.

Table 6.3: Observed weaknesses of manual counting and the corresponding features

Weakness at the packing table	Feature of the component
Counted every 15 minutes, copied later	Each garment recorded within seconds with a server timestamp
Two error prone manual steps, no audit trail	Automatic capture with the garment image stored as evidence
One total for a mixed order	Counts, targets and forecasts per style, plus colours
Figures too late to act on	Live projection and overtime advice; efficiency in the decision summary
Breakdowns make the line look slow	Rates measured against effective hours

6.4 Integration results

In the bench trial the integrated system produced 37 captures in five and a half minutes, all flowing from camera to dashboard without error (Figure 6.7). There were 20 T shirts, 15 shirts, 1 trouser and 1 short. Mean accepted confidence was 89.12 % and the lowest 85.09 %, so no record below 0.85 was stored; 20 captures fell between 0.85 and 0.90, 16 between 0.90 and 0.95 and one above. Main colours were black (13), beige (8), navy (7), brown (4), grey (3) and white (2). The logged downtime was accepted, reduced the day's effective hours and stopped the camera as designed. The trial therefore exercised every layer of the component in sequence: frame acquisition, the six gates, colour extraction, the asynchronous post, validation and storage in the data service, the downtime logic in the analytics, and the live update of the dashboard. Without an independent count, the trial shows end to end operation and threshold enforcement, not counting accuracy.

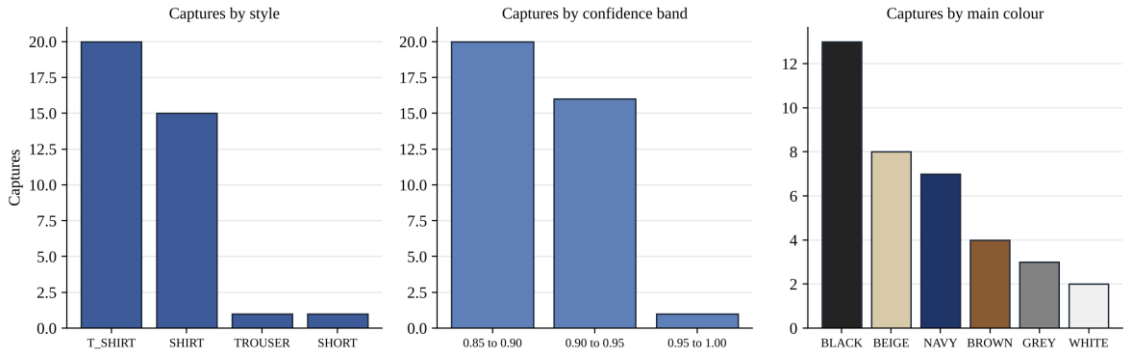


Figure 6.7: Bench trial captures by style, confidence band and main colour

6.5 Summary of findings

- The detector reached test precision 0.962, recall 0.955 and mAP@0.5 0.958 at about 27 ms per frame on a CPU (RQ1: yes).
- The benchmark missed a full frame box failure and a confident false positive on a face, both explained by tightly cropped training images with no negatives (RQ2: benchmark mAP overstated field reliability).
- Six interpretable gates stopped the observed failures without retraining, at a deliberate cost in recall (RQ3: yes, on qualitative evidence).
- The colour method named test colours correctly and gave every bench capture a main colour (RQ4: yes, within the limits of synthetic tests and a short trial).

- The analytics separated downtime from pace and gave correct figures on the worked example; one forecast defect of about 1.5 % was found (RQ5: yes).

7 DISCUSSION

7.1 Interpretation of findings

The most important result is the distance between the detector's accuracy and its behaviour at the table. A test mAP@0.5 of 0.958 suggests the problem is solved; the live system showed otherwise within minutes. The reason lies in what the training and test images share. Every image is a tight crop of one garment with a box covering nearly the whole frame, so the model learns two things together: how to tell the styles apart, and that the correct box is almost always the full frame. The second lesson is a shortcut in the sense of Geirhos et al. [12], valid inside the dataset and wrong outside it, and a test split with the same framing rewards it instead of exposing it.

The face and hair false positive tells a related story. A four class model with no rejection class cannot say that an input is none of the things it knows, so it chooses the nearest class and reports a confident score. This is a structural property of closed set models [40], whose scores are not calibrated probabilities of being right [42].

Seen this way, each gate adds knowledge the model lacks: that a folded garment does not fill the zone, what the empty table looks like, that a good record needs a settled and clear image, and that a real garment stays in view for several frames. Only the confidence gate relies on the model's own judgement, and its threshold came from evidence gathered at the table, not from the benchmark. Raising the threshold from the F1 optimum of 0.665 to 0.85 costs recall by the benchmark's accounting, while its benefit, rejecting a face, scores nothing because the test set contains no faces. The benchmark can price the cost of the acceptance policy but not its benefit, which is perhaps the clearest illustration of why benchmarks alone are a poor guide to deploying a closed set detector.

The bench trial supports this reading in a modest way. Every accepted capture had a confidence of at least 85.09 %, so the threshold was enforced in practice, and the mean of 89.12 % shows that genuine garments cleared it with room to spare under the trial conditions. At the same time, the trial was too short and too narrow in colour and style to show how often real garments fall just below the threshold, which is the

cost side of the precision over recall choice and the main thing a field trial must measure.

The analytics show that a low count is not necessarily a slow day, and the overtime recommendation turns measurement into a decision, the step manual tallies never reached.

7.2 Comparison with literature baselines and existing solutions

The detector's speed and accuracy fit the general picture for nano scale YOLO models [3], [5]. Direct comparison with benchmarks such as DeepFashion2 [7] would mislead, since those involve more classes, worn garments and full scene localisation; the present task is easier, and much easier in localisation than its mAP suggests. The overstatement of real performance agrees with Torralba and Efros [11] and Recht et al. [13]; this work adds an industrial case where the mechanism can be traced precisely. The finding that reliability came from the code around the model echoes Sculley et al. [43]. For colour, seeding GrabCut [29] with a strong core prior and adding the Chai and Ngan skin map [30] address the intrusions typical of a packing table, and CIELAB [31] removed the special case for red. The analytics also differ from common practice in production reporting. Many dashboards report pieces per day or per clock hour, which silently blends availability losses with performance, the very confusion that OEE was designed to avoid [14]. Measuring against effective hours keeps the two apart, and the simple, inspectable forecast follows the advice of Hyndman and Athanasopoulos [38] that short series are better served by transparent methods than by complex models fitted to a handful of points.

Against the alternatives in Table 2.2, the component counts by style and colour without tags or changes to the operator's routine, on hardware a factory already owns.

7.3 Research contribution

First, a documented case showing that benchmark mAP overstates field reliability for a closed set garment detector trained on tightly cropped images, with the two responsible mechanisms identified: a framing shortcut and the absence of a negative class. Second, a demonstration that a cascade of interpretable gates, each tied to an observed failure and an evidenced threshold, closes those failures without a larger

model. Third, an explanation of why the benchmark's preferred operating point differs from the right deployment threshold.

7.4 Product contribution

The component is a working end to end system that recognises garments in real time on a CPU, stores each one once with style, colours, confidence and image, and turns the records into downtime aware metrics and overtime advice for the order and each category. It starts with one command, guides camera setup, shows its own uncertainty, refuses unsupported estimates, and is covered by 107 passing tests.

7.5 Limitations and threats to validity

- **Benchmark validity.** The dataset's framing limits what the reported mAP says about any differently framed deployment.
- **Dataset.** 3,033 images, four classes, no negatives, and a realised split different from the configured one.
- **Qualitative gate evidence.** The gates' effect rests on observed failures, not a controlled count over long sessions, and the 0.82 versus 0.75 separation may not hold under other lighting, cameras or garments.
- **Short bench trial.** Five and a half minutes, 37 mostly dark or neutral captures, and no independent count, so no accuracy, miss rate or labour saving claims.
- **Approximate recall.** Recall at 0.85 was read from a plot.
- **Assumptions and defects.** The empty table assumption at start up, one garment and one station at a time, the reversed forecast weights and a misleading function name.
- **Engineering.** No authentication, development servers, a perception service that listens on all interfaces, no retry queue for failed posts, no pagination, inline images and local time stamps; acceptable for one workstation only.

Taken together, these limitations mean that the thesis supports strong claims about the model's benchmark behaviour, the correctness of the analytics and the existence of the observed failures, but only cautious claims about field counting accuracy. The field trial described in Chapter 9 is the step that would turn the qualitative evidence for the gates into measured rates.

7.6 Implications

For apparel manufacturers, automatic counting at the packing table is achievable with inexpensive hardware if the detector is surrounded by checks that reflect the workstation, and its real value lies in a fair efficiency figure, a projected completion date and concrete overtime advice. For engineers deploying detectors, a test split drawn from the training distribution checks learning, not deployment readiness: when a closed set model will meet unknown inputs, evaluation should include them, training data should include a background class, and thresholds should be chosen with deployment inputs in view. For this component, the next gain in reliability will come from better data rather than a bigger model.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The component turns an ordinary USB camera and an existing factory computer into a live counting and planning instrument: instead of learning tomorrow that an order fell behind today, a supervisor learns it within the hour, along with the overtime needed to recover. Table 8.1 shows who benefits. The buyers would be apparel manufacturers, especially small and medium plants that cannot justify tagging every garment or a full manufacturing execution system.

Table 8.1: Benefits of the component for each user group

User group	Benefit
Production supervisor	Live progress, fair efficiency, projected completion and overtime advice
Line manager	Progress and forecasts per style and colour
Packing operator	No tally sheet; clear on screen status; quick downtime logging
Factory management	Audit trail with an image of every counted garment

8.2 Market and competitive position

The apparel sector is a major Sri Lankan export industry [1] with many factories and packing stations, and the same need exists across the region. Alternatives are manual tallies (no equipment cost but slow and error prone), barcode or RFID tracking (accurate but needing tags, readers and a scanning step), production planning software (dependent on manual or sensor counts) and custom vision systems built per site. The component's differentiators are no tagging or change to the operator's routine, hardware the factory already owns, downtime aware analytics with concrete overtime advice, and full transparency, with every threshold explainable and every capture backed by an image.

8.3 Business or sustainability model

A suitable model is a modest installation fee covering camera mounting, zone setup and supervisor training, plus a subscription per station covering updates, retraining for new styles and support. Because the software runs on the customer's own computer, each extra station costs little to serve, so pricing per station scales with

customer size. Table 8.2 outlines the cost structure of one station; actual prices would depend on local market research outside the scope of this thesis.

Table 8.2: Indicative cost structure for one packing station

Item	Nature of cost
Camera and mount	One time, low cost consumer hardware
Workstation	Usually an existing PC; no graphics card needed
Installation and setup	One time service per station
Software subscription	Recurring, per station
Retraining for new styles	Included in subscription or charged per style set

A pilot on one line, run alongside the manual count for several weeks, would supply the independent ground truth this thesis lacks, measure accuracy and time saved, and provide a case study for further sales.

8.4 Scalability intellectual property and compliance

Scaling to a whole factory would mean a perception service per station sending records to a central data service on a server database such as PostgreSQL; since all SQL sits in the data service, the perception code would not change. Authentication, a production web server, pagination and a retry queue would be added at that stage. The Ultralytics framework is released under the AGPL 3.0 licence with an enterprise licence available for commercial products [3], so a commercial release would either comply with AGPL or obtain that licence, and other dependencies would be reviewed similarly. Workplace cameras raise surveillance concerns; storing only garment crops, processing locally, measuring per station rather than per person, and excluding downtime from rates address these, and a deployment would also need worker consultation, signage and compliance with Sri Lanka's Personal Data Protection Act and buyers' social compliance audits.

8.5 Contribution to the Sustainable Development Goals

The component supports several Sustainable Development Goals [48]. For SDG 8 (decent work), it removes repetitive tallying and gives a fairer measure of performance, letting overtime be planned rather than imposed at the last minute. For SDG 9 (industry and innovation), it brings affordable digital tools to small and medium manufacturers. For SDG 12 (responsible production), timely counts by style

reduce over production and wasteful last minute rushes. Running on modest hardware without a graphics processor also keeps its energy use and cost low.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis replaced the delayed manual counting of garments at an apparel packing table with a camera based component that recognises style and colour, records each garment only on trustworthy evidence, and turns the records into decisions a supervisor can use during the shift. The YOLOv8 nano detector reached test precision 0.962, recall 0.955 and mAP@0.5 0.958 at about 27 ms per frame on a CPU, showing that real time recognition is feasible on inexpensive hardware. The central lesson, however, was that these numbers did not describe live behaviour: trained on tightly cropped images without negatives, the detector boxed whole frames and labelled a face as a shirt, failures the benchmark could not reveal. They were closed by six interpretable gates, a capture lock and a visible UNCERTAIN state rather than a larger model. Colour was measured from garment pixels alone, and the analytics separated downtime from pace to deliver efficiency, projections, overtime advice and forecasts. All 107 tests passed and a bench trial confirmed end to end operation.

9.2 Achievement of objectives

Table 9.1: Achievement of the research objectives

Objective	Summary	Evidence
O1	Lightweight detector on CPU	Test mAP@0.5 0.958; about 27 ms per frame (6.1.1, 6.2)
O2	Evidence based capture mechanism	Six gate cascade and capture lock (4.4.2, 6.2)
O3	Colour from garment pixels	Segmentation, skin removal, CIELAB clustering; tests and trial (4.4.4, 6.2)
O4	Downtime aware analytics	Worked example verified by hand and tests (6.1.3)
O5	Integrated, verified application	107 of 107 tests passed; bench trial (5.4, 6.4)
O6	Benchmark versus field analysis	Framing shortcut and closed set behaviour explained (6.2, 7.1)

All six objectives were met (Table 9.1), with the qualification that the gates' effect and integrated behaviour were shown qualitatively and in a short trial rather than in a long field evaluation.

9.3 Individual research and product contributions

The research contribution is the identification and explanation of the gap between benchmark accuracy and field reliability for a closed set garment detector trained on cropped images, the demonstration that evidence based gates close the observed failures, and the analysis of why the benchmark's optimal threshold is wrong for deployment. The product contribution is the working component: detector, perception service, colour method, data service and analytics, dashboard and test suite.

Beyond its specific results, the work leaves a stable base for further development. The record format, the endpoint contract and the shared class vocabulary allow new features, such as additional event sources or a detected folding zone, to connect without changing the existing component, and the test suite gives future contributors a safety net when they extend it.

9.4 Future directions

- Collect training data from the deployment camera position, including empty tables, hands, tools and people, with a background class.
- Run a multi week field trial against an independent manual count to measure accuracy, miss rate and time saved.
- Add open set rejection and calibration [41], [42].
- Detect the folding zone automatically instead of using a fixed region, and allow stoppages to be logged from a simple device at the station as well as from the dashboard.
- Fix the forecast weight order, rename the current rate function, and add authentication, a retry queue, pagination and file based images.
- Extend recognition to more styles and to patterns, and evaluate forecast accuracy once months of data exist.

Trustworthy automation at the end of an apparel line does not need expensive hardware or very large models. It needs a clear view of what the model has and has not learned, and simple checks that reflect the place where it works.

REFERENCES

[1] Sri Lanka Export Development Board, "Apparel and textile industry in Sri Lanka," [Online]. Available: https://www.srilankabusiness.com/apparel/

[2] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, "Industry 4.0," *Business & Information Systems Engineering*, vol. 6, no. 4, pp. 239–242, 2014.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[5] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.

[6] Z. Liu, P. Luo, S. Qiu, X. Wang, and X. Tang, "DeepFashion: Powering robust clothes recognition and retrieval with rich annotations," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.

[7] Y. Ge, R. Zhang, X. Wang, X. Tang, and P. Luo, "DeepFashion2: A versatile benchmark for detection, pose estimation, segmentation and re-identification of clothing images," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5337–5345.

[8] K. Yamaguchi, M. H. Kiapour, L. E. Ortiz, and T. L. Berg, "Parsing clothing in fashion photographs," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3570–3577.

[9] L. Bossard, M. Dantone, C. Leistner, C. Wengert, T. Quack, and L. Van Gool, "Apparel classification with style," in *Proc. Asian Conf. Computer Vision (ACCV)*, 2012, pp. 321–335.

[10] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Industrial Electronics*, vol. 55, no. 1, pp. 348–363, 2008.

[11] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2011, pp. 1521–1528.

[12] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.

[13] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do ImageNet classifiers generalize to ImageNet?" in *Proc. Int. Conf. Machine Learning (ICML)*, 2019, pp. 5389–5400.

[14] S. Nakajima, *Introduction to TPM: Total Productive Maintenance*. Cambridge, MA, USA: Productivity Press, 1988.

[15] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[17] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[19] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[20] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1314–1324.

[21] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 91–99.

[22] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. European Conf. Computer Vision (ECCV)*, 2016, pp. 21–37.

[23] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[24] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475.

[25] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes (VOC) challenge," *Int. J. Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.

[26] R. Padilla, S. L. Netto, and E. A. B. da Silva, "A survey on performance metrics for object-detection algorithms," in *Proc. Int. Conf. Systems, Signals and Image Processing (IWSSIP)*, 2020, pp. 237–242.

[27] J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez, and J. Fernández-Valdivia, "Diatom autofocusing in brightfield microscopy: A comparative study," in *Proc. 15th Int. Conf. Pattern Recognition (ICPR)*, vol. 3, 2000, pp. 314–317.

[28] S. Pertuz, D. Puig, and M. A. Garcia, "Analysis of focus measure operators for shape-from-focus," *Pattern Recognition*, vol. 46, no. 5, pp. 1415–1432, 2013.

[29] C. Rother, V. Kolmogorov, and A. Blake, "GrabCut: Interactive foreground extraction using iterated graph cuts," *ACM Trans. Graphics*, vol. 23, no. 3, pp. 309–314, 2004.

[30] D. Chai and K. N. Ngan, "Face segmentation using skin-color map in videophone applications," *IEEE Trans. Circuits and Systems for Video Technology*, vol. 9, no. 4, pp. 551–564, 1999.

[31] M. D. Fairchild, *Color Appearance Models*, 3rd ed. Chichester, U.K.: Wiley, 2013.

[32] G. Sharma, W. Wu, and E. N. Dalal, "The CIEDE2000 color-difference formula: Implementation notes, supplementary test data, and mathematical observations," *Color Research & Application*, vol. 30, no. 1, pp. 21–30, 2005.

[33] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[34] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

[35] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022.

[36] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

[37] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[38] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[39] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," arXiv:1708.07747, 2017.

[40] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boult, "Toward open set recognition," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 35, no. 7, pp. 1757–1772, 2013.

[41] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[42] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.

[43] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 2503–2511.

[44] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," arXiv:1606.06565, 2016.

[45] Electron, "Context isolation," Electron Documentation. [Online]. Available: https://www.electronjs.org/docs/latest/tutorial/context-isolation

[46] SQLite, "Write-ahead logging," SQLite Documentation. [Online]. Available: https://www.sqlite.org/wal.html

[47] S. Ramírez, "FastAPI," FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/

[48] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

GLOSSARY

Term	Definition
Bench trial	A short test of the complete system under controlled conditions, used here to show end to end operation rather than to measure accuracy.
Capture gate	One of the independent checks a detection must pass before it is recorded as a garment.
Closed set detector	A model that can only answer with one of a fixed list of classes and has no option to say that an input belongs to none of them.
Confidence	The score between 0 and 1 that the detector attaches to a prediction; it indicates how strongly the model favours a class but is not a calibrated probability.
Consensus window	The rolling record of the last four attempts, from which a capture is triggered when at least two are good.
Counting cycle	The period since the count_since marker over which packed garments are credited to the current target.
Effective hours	Scheduled shift hours less fixed breaks and less any logged downtime that overlaps the day.
Efficiency	In this thesis, the current rate divided by the rate the order requires, expressed as a percentage.
Folding zone	The region of each frame, drawn on the live feed, within which garments are detected.
GrabCut	An iterative segmentation method that separates foreground from background using colour mixture models and a graph cut.
k means	A clustering algorithm that groups points around k centres by alternately assigning points to the nearest centre and moving each centre to the mean of its points.
Laplacian	A second derivative image filter that responds strongly at edges; the variance of its response measures sharpness.
Shortcut learning	The tendency of a model to rely on a cue that predicts the label in the training data but does not hold in general.
Singleton table	A database table constrained to hold at most one row.
Transfer learning	Fine tuning a model pretrained on a large general dataset for a smaller specific task.

APPENDICES

The appendices provide supplementary material referred to in the main text.

Appendix A Complete list of automated test cases

Table A.1 lists every test in the automated suite described in Section 5.4, with the behaviour it verifies. All 107 tests passed when the suite was executed, so the outcome of every case listed is Pass.

Table A.1: Automated test cases and outcomes

ID	Behaviour verified	ID	Behaviour verified
UT_A01	Parse hhmm	UT_V10	Blur threshold is the documented value
UT_A02	Scheduled hours normal shift	UT_V11	Identical crops score zero difference
UT_A03	Scheduled hours never negative	UT_V12	Different crops score a large difference
UT_A04	Fixed break hours	UT_V13	Similarity compares crops of different sizes
UT_A05	No breaks configured	UT_V14	Similarity handles empty input
UT_A06	Downtime overlap inside one day	UT_V15	Colour difference is not collapsed to brightness
UT_A07	Downtime spanning midnight splits across both days	UT_V16	Inset crop trims the requested margin
UT_A08	Downtime not overlapping day is zero	UT_V17	Inset crop refuses to shrink a tiny crop
UT_A09	Downtime capped by until	UT_V18	Skin mask selects skin tone and rejects navy
UT_A10	Multiple downtime events sum	UT_V19	Clean mask keeps only the largest component
UT_A11	Effective hours past day full shift	UT_V20	Segment garment returns a mask matching the working image
UT_A12	Effective hours past day minus downtime	UT_V21	Solid navy is named navy
UT_A13	Effective hours today capped to elapsed	UT_V22	Solid red is named red
UT_A14	Effective hours today counts only started breaks	UT_V23	Solid white is named white
UT_A15	Effective hours never negative	UT_V24	Solid black is named black
UT_A16	Weighted recent rate mean of last three	UT_V25	Dominant colour is reported first
UT_A17	Weighted recent rate skips unreliable days	UT_V26	Noise clusters below the floor are discarded
UT_A18	Weighted recent rate empty history is zero	UT_V27	Palette has the documented fifteen colours
UT_A19	Min reliable hours boundary	UT_V28	Colour extraction degrades to unknown not a crash
UT_A20	Summary core figures	UT_V29	Capture gate constants are the documented values
UT_A21	Estimated days rounds up	UT_V30	Consensus window accepts two good of four non consecutive
UT_A22	No rate history returns none not a guess	UT_V31	Consensus window forgets beyond four attempts
UT_A23	Overtime recommendation	UT_V32	New gate constants exist with documented values
UT_A24	No overtime when on track	UT_V33	Empty table snapshot is rejected as background
UT_A25	Counter freezes at target and reports overrun	UT_V34	A garment on the table is not rejected as background
UT_A26	Forecast no data	UT_V35	A still garment passes the stillness gate
UT_A27	Forecast uses moving average only below	UT_V36	A sliding garment fails the stillness gate

ID	Behaviour verified	ID	Behaviour verified
	three days		
UT_A28	Forecast blends trend from three days	IT_01	Service is up
UT_A29	Forecast window is last five days	IT_02	Settings returns 404 before configuration
UT_A30	Forecast rate is never negative	IT_03	Summary returns 404 before configuration
UT_A31	Forecast weight vector is applied oldest first	IT_04	Camera returns 404 before configuration
UT_D01	All four tables created	IT_05	Malformed body is rejected by validation
UT_D02	Both indexes created	IT_06	Invalid downtime type is rejected
UT_D03	Garments required columns are not null	IT_07	Capture is persisted and readable
UT_D04	Settings singleton rejects a second row	IT_08	Timestamp is assigned by the server
UT_D05	Device settings singleton rejects a second row	IT_09	Latest returns null on an empty database
UT_D06	Downtime type check rejects unknown value	IT_10	History is returned most recent first
UT_D07	Downtime type accepts both valid values	IT_11	Delete all reports the number removed
UT_D08	Write ahead logging is enabled	IT_12	Sql injection attempt is stored as data
UT_D09	Foreign keys enforced	IT_13	Settings round trip
UT_D10	Rows behave like mappings	IT_14	Saving settings starts a new counting cycle
UT_D11	Transaction is not committed when the block raises	IT_15	Upsert updates in place and never creates a second row
UT_D12	Migration is idempotent	IT_16	Reset progress moves the baseline without deleting records
UT_D13	Migration upgrades a legacy settings table	IT_17	Camera selection round trip
UT_D14	Removed roi settings table is dropped	IT_18	Downtime end must be after start
UT_V01	Roi bounds match the documented fractions	IT_19	Downtime is logged and filtered by day
UT_V02	Roi area gate is 15 to 85 percent	IT_20	Daily counts group by date
UT_V03	Roi scales with frame size	IT_21	Summary produces the full decision bundle
UT_V04	Full frame box is rejected by the area gate	IT_22	Downtime is reflected in the daily rate
UT_V05	A plausible garment box passes the area gate	IT_23	All four categories are reported
UT_V06	Sharp image passes the blur gate	IT_24	Forecast reports its own method and window
UT_V07	Blurred image fails the blur gate	IT_25	Forecast declines to guess without history
UT_V08	Flat image fails the blur gate	IT_26	Counter freezes once the target is reached
UT_V09	Blur gate handles empty input		

Appendix B Academic integrity and AI use disclosure

During the development of this research project, artificial intelligence tools, primarily ChatGPT and Claude were used as technical aids. Their use was focused on development, debugging and problem solving workflows.

Debugging and Error Resolution: AI tools served as a troubleshooting assistant to diagnose build errors, runtime exceptions and stack traces encountered across the Python (Flask and FastAPI) and Electron and React development environments.

Algorithm and Architecture Refinement: When addressing camera challenges, such as motion blur, lighting variance and boundary false positives AI suggestions

were consulted to explore conceptual approaches. This helped in the iterative design of the capture gate verification steps.

Code Optimisation and Scripting Support: AI was consulted to help structure utility scripts for dataset splitting and formatting. It also assisted with reviewing expressions API routing patterns and automated unit test cases.

Technical Concept Clarification: Conversational queries were used to explore practices, in CIELAB colour space processing, model evaluation metrics (precision, recall, mAP) and asynchronous service handling.

All final code implementations, dataset collection, pipeline thresholds (including the 0.85 confidence filter) experimental evaluations and analytical conclusions reflect the work of the author. All outputs were manually tested, verified and integrated within the project environment.